

# Link.Foundation.Links.Notations Class Library

## 1.1 ./Link.Foundation.Links.Notations/Comments.cs

```
1 namespace Link.Foundation.Links.Notations
2 {
3     /// <summary>
4     /// Comments, and the rule that decides where one starts.
5     /// </summary>
6     /// <remarks>
7     /// <c>#</c> starts a comment when it opens a token: at the start of the document or
8     /// after a space, a tab or a line break. A <c>#</c> written inside a token
9     /// (<c>issue#1047</c>) or inside a delimited reference (<c>"# not a comment"</c>) is
10    /// an ordinary character.
11    /// </remarks>
12    public static class Comments
13    {
14        /// <summary>
15        /// The character that starts a comment.
16        /// </summary>
17        public const char Comment = '#';
18
19        /// <summary>
20        /// What a delimited reference may start after.
21        /// </summary>
22        private const string BeforeReference = " \t\n\r(";
23
24        /// <summary>
25        /// What a comment may start after.
26        /// </summary>
27        private const string BeforeComment = " \t\n\r";
28
29        /// <summary>
30        /// Blanks every comment of a document.
31        /// </summary>
32        /// <param name="document">The document to read.</param>
33        /// <returns>
34        /// The same document with the characters of every comment replaced by spaces.
35        /// </returns>
36        /// <remarks>
37        /// Every character of a comment is replaced rather than removed, so every later
38        /// character keeps the position it had in the document the caller wrote, and
39        /// anything the parser reports about a position still points at what the reader
40        /// can see.
41        /// </remarks>
42        public static string StripComments(string document)
43        {
44            char[]? blanked = null;
45            int position = 0;
46
47            while (position < document.Length)
48            {
49                char character = document[position];
50
51                if (Quotes.IsQuote(character) && Follows(document, position, BeforeReference))
52                {
53                    int end = Quotes.ReferenceEnd(document, position);
54                    position = end < 0 ? position + 1 : end;
55                    continue;
56                }
57
58                if (character == Comment && Follows(document, position, BeforeComment))
59                {
60                    blanked ??= document.ToCharArray();
61                    while (position < document.Length &&
62                        document[position] != '\n' &&
63                        document[position] != '\r')
64                    {
65                        blanked[position] = ' ';
66                        position++;
67                    }
68                    continue;
69                }
70
71                position++;
72            }
73
74            return blanked == null ? document : new string(blanked);
75        }
76    }
77 }
```

```

78     /// <summary>
79     /// Reports whether the character before the position is one of the allowed ones,
80     /// the start of the document counting as allowed.
81     /// </summary>
82     private static bool Follows(string document, int position, string allowed) =>
83         position == 0 || allowed.IndexOf(document[position - 1]) >= 0;
84 }
85 }

```

## 1.2 ./Link.Foundation.Links.Notation/FormatConfig.cs

```

1 namespace Link.Foundation.Links.Notation
2 {
3     /// <summary>
4     /// FormatConfig is an alias for FormatOptions to maintain API consistency
5     /// with the Python implementation.
6     ///
7     /// C# uses 'FormatOptions' internally, but provides 'FormatConfig'
8     /// as an alias to match Python's naming convention.
9     /// </summary>
10    public class FormatConfig : FormatOptions
11    {
12        // FormatConfig inherits all functionality from FormatOptions
13        // No additional implementation needed - this is purely for naming consistency
14    }
15 }

```

## 1.3 ./Link.Foundation.Links.Notation/FormatOptions.cs

```

1 using System;
2
3 namespace Link.Foundation.Links.Notation
4 {
5     /// <summary>
6     /// FormatOptions for Lino notation formatting.
7     /// Provides configuration options for controlling how Link objects are formatted.
8     /// </summary>
9     public class FormatOptions
10    {
11        /// <summary>
12        /// If true, omit parentheses where safe (default: false)
13        /// </summary>
14        public bool LessParentheses { get; set; } = false;
15
16        /// <summary>
17        /// Maximum line length before auto-indenting (default: 80)
18        /// </summary>
19        public int MaxLineLength { get; set; } = 80;
20
21        /// <summary>
22        /// If true, indent lines exceeding MaxLineLength (default: false)
23        /// </summary>
24        public bool IndentLongLines { get; set; } = false;
25
26        /// <summary>
27        /// Maximum number of references before auto-indenting (default: null = unlimited)
28        /// </summary>
29        public int? MaxInlineRefs { get; set; } = null;
30
31        /// <summary>
32        /// If true, group consecutive links with same ID (default: false)
33        /// </summary>
34        public bool GroupConsecutive { get; set; } = false;
35
36        /// <summary>
37        /// String to use for indentation (default: "  " = two spaces)
38        /// </summary>
39        public string IndentString { get; set; } = "  ";
40
41        /// <summary>
42        /// If true, prefer inline format when under thresholds (default: true)
43        /// </summary>
44        public bool PreferInline { get; set; } = true;
45
46        /// <summary>
47        /// Check if line should be indented based on length
48        /// </summary>
49        /// <param name="line">The line to check</param>
50        /// <returns>True if line should be indented based on length threshold</returns>
51        public bool ShouldIndentByLength(string line)

```

```

52     {
53         if (!IndentLongLines)
54         {
55             return false;
56         }
57         // Count printable unicode characters
58         return line.Length > MaxLineLength;
59     }
60
61     /// <summary>
62     /// Check if link should be indented based on reference count
63     /// </summary>
64     /// <param name="refCount">Number of references in the link</param>
65     /// <returns>True if link should be indented based on reference count threshold</returns>
66     public bool ShouldIndentByRefCount(int refCount)
67     {
68         if (MaxInlineRefs == null)
69         {
70             return false;
71         }
72         return refCount > MaxInlineRefs.Value;
73     }
74 }
75 }

```

#### 1.4 ./Link.Foundation.Links.Notation/FurthestFailureTracer.cs

```

1  using Pegasus.Common;
2  using Pegasus.Common.Tracing;
3
4  namespace Link.Foundation.Links.Notation
5  {
6      /// <summary>
7      /// Remembers the furthest position a parse reached.
8      /// </summary>
9      /// <remarks>
10     /// A parsing expression grammar backtracks, so the cursor the generated parser raises
11     /// its error with is the start of the document rather than the place the document
12     /// stopped making sense. The furthest position any rule reached is that place, and the
13     /// tracer is the only hook the generated parser offers to observe it.
14     /// </remarks>
15     internal sealed class FurthestFailureTracer : ITracer
16     {
17         /// <summary>Offset of the furthest position the parse reached.</summary>
18         public int Furthest { get; private set; }
19
20         public void TraceRuleEnter(string ruleName, Cursor cursor) => Reach(cursor.Location);
21
22         public void TraceRuleExit<T>(string ruleName, Cursor cursor, IParseResult<T> result)
23         {
24             if (result != null) Reach(result.EndCursor.Location);
25         }
26
27         public void TraceCacheHit<T>(string ruleName, Cursor cursor, CacheKey key,
↪ IParseResult<T> result)
28         {
29             if (result != null) Reach(result.EndCursor.Location);
30         }
31
32         public void TraceCacheMiss(string ruleName, Cursor cursor, CacheKey key)
33         {
34         }
35
36         public void TraceInfo(string ruleName, Cursor cursor, string info)
37         {
38         }
39
40         private void Reach(int location)
41         {
42             if (location > Furthest) Furthest = location;
43         }
44     }
45 }

```

#### 1.5 ./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs

```

1  using System.Collections.Generic;
2  using System.Runtime.CompilerServices;
3
4  namespace Link.Foundation.Links.Notation
5  {

```

```

6     /// <summary>
7     /// Provides extension methods for collections of <see cref="LinksGroup{TLinkAddress}"/>
↪ instances.
8     /// </summary>
9     public static class ILinksGroupListExtensions
10    {
11        /// <summary>
12        /// Converts a collection of links groups to a flat list of links by processing each
↪ group hierarchically.
13        /// </summary>
14        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This
↪ can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
15        /// <param name="groups">The collection of links groups to convert.</param>
16        /// <returns>A flat list of links representing all the links from the input
↪ groups.</returns>
17        [MethodImpl(MethodImplOptions.AggressiveInlining)]
18        public static List<Link<TLinkAddress>> ToLinksList<TLinkAddress>(this
↪ IList<LinksGroup<TLinkAddress>> groups)
19        {
20            var list = new List<Link<TLinkAddress>>();
21            for (var i = 0; i < groups.Count; i++)
22            {
23                CollectLinksWithIndentedIdSyntaxSupport(list, groups[i], null);
24            }
25            return list;
26        }
27
28        private static void
↪ CollectLinksWithIndentedIdSyntaxSupport<TLinkAddress>(List<Link<TLinkAddress>> list,
↪ LinksGroup<TLinkAddress> group, Link<TLinkAddress>? parentDependency)
29        {
30            var link = group.Link;
31            var groups = group.Groups;
32
33            if (groups != null && groups.Count > 0)
34            {
35                bool isIndentedIdSyntax = link.Id != null &&
36                                         (link.Values == null || link.Values.Count == 0);
37
38                if (isIndentedIdSyntax && !parentDependency.HasValue)
39                {
40                    var childValues = new List<Link<TLinkAddress>>();
41                    for (int i = 0; i < groups.Count; i++)
42                    {
43                        var transformedLink = TransformIndentedIdLink(groups[i]);
44                        childValues.Add(transformedLink);
45                    }
46
47                    var linkWithChildren = new Link<TLinkAddress>(link.Id, childValues);
48                    list.Add(linkWithChildren);
49                }
50                else
51                {
52                    var transformedGroups = new List<LinksGroup<TLinkAddress>>();
53                    for (int i = 0; i < groups.Count; i++)
54                    {
55                        var childGroup = groups[i];
56                        var childLink = childGroup.Link;
57                        var childGroups = childGroup.Groups;
58
59                        if (childLink.Id != null &&
60                            (childLink.Values == null || childLink.Values.Count == 0) &&
61                            childGroups != null && childGroups.Count > 0)
62                        {
63                            var transformedLink = TransformIndentedIdLink(childGroup);
64                            transformedGroups.Add(new LinksGroup<TLinkAddress>(transformedLink));
65                        }
66                        else
67                        {
68                            transformedGroups.Add(childGroup);
69                        }
70                    }
71
72                    var currentDependency = parentDependency.HasValue ?
↪ parentDependency.Value.Combine(link) : link;
73                    list.Add(currentDependency);
74
75                    for (int i = 0; i < transformedGroups.Count; i++)
76                    {

```

```

77         CollectLinksWithIndentedIdSyntaxSupport(list, transformedGroups[i],
↪     currentDependency);
78     }
79 }
80 }
81 else
82 {
83     var currentLink = parentDependency.HasValue ?
↪     parentDependency.Value.Combine(link) : link;
84     list.Add(currentLink);
85 }
86 }
87
88 private static Link<TLinkAddress>
↪ TransformIndentedIdLink<TLinkAddress>(LinksGroup<TLinkAddress> group)
89 {
90     var link = group.Link;
91     var groups = group.Groups;
92
93     if (groups != null && groups.Count > 0 &&
94         link.Id != null &&
95         (link.Values == null || link.Values.Count == 0))
96     {
97         var childValues = new List<Link<TLinkAddress>>();
98         for (int i = 0; i < groups.Count; i++)
99         {
100             var transformedChild = TransformIndentedIdLink(groups[i]);
101             childValues.Add(transformedChild);
102         }
103         return new Link<TLinkAddress>(link.Id, childValues);
104     }
105     else if (groups != null && groups.Count > 0)
106     {
107         var values = new List<Link<TLinkAddress>>();
108         if (link.Values != null)
109         {
110             values.AddRange(link.Values);
111         }
112         for (int i = 0; i < groups.Count; i++)
113         {
114             values.Add(TransformIndentedIdLink(groups[i]));
115         }
116         return new Link<TLinkAddress>(link.Id, values);
117     }
118     else if (!group.IsParenthesized && link.Id == null && link.Values != null &&
↪     link.Values.Count == 1)
119     {
120         return link.Values[0];
121     }
122     else
123     {
124         return link;
125     }
126 }
127 }
128 }

```

## 1.6 ./Link.Foundation.Links.Notation/IListExtensions.cs

```

1 using Platform.Collections;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Runtime.CompilerServices;
6
7 namespace Link.Foundation.Links.Notation
8 {
9     /// <summary>
10     /// Provides extension methods for formatting collections of <see
↪     cref="Link{TLinkAddress}" /> instances.
11     /// </summary>
12     public static class IListExtensions
13     {
14         /// <summary>
15         /// Formats a collection of links as a multi-line string with each link on a separate
↪     line.
16         /// </summary>
17         /// <typeparam name="TLinkAddress">The type used for link
↪     addresses/identifiers.</typeparam>
18         /// <param name="links">The collection of links to format.</param>

```

```

19     /// <returns>A multi-line string representation of the links.</returns>
20     [MethodImpl(MethodImplOptions.AggressiveInlining)]
21     public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links) =>
↪     string.Join(Environment.NewLine, links.Select(l => l.ToString()));
22
23     /// <summary>
24     /// Formats a collection of links as a multi-line string with optional parentheses
↪     trimming for cleaner output.
25     /// </summary>
26     /// <typeparam name="TLinkAddress">The type used for link
↪     addresses/identifiers.</typeparam>
27     /// <param name="links">The collection of links to format.</param>
28     /// <param name="lessParentheses">True to remove outer parentheses from each link for
↪     cleaner formatting; false to use standard formatting.</param>
29     /// <returns>A multi-line string representation of the links.</returns>
30     [MethodImpl(MethodImplOptions.AggressiveInlining)]
31     public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links, bool
↪     lessParentheses)
32     {
33         if (lessParentheses == false)
34         {
35             return links.Format();
36         }
37         else
38         {
39             ↪     return string.Join(Environment.NewLine, links.Select(l =>
↪     l.ToString().TrimSingle('(').TrimSingle(')')));
40         }
41     }
42
43     /// <summary>
44     /// Formats a collection of links using FormatOptions configuration.
45     /// </summary>
46     /// <typeparam name="TLinkAddress">The type used for link
↪     addresses/identifiers.</typeparam>
47     /// <param name="links">The collection of links to format.</param>
48     /// <param name="options">The FormatOptions to use for formatting.</param>
49     /// <returns>A multi-line string representation of the links.</returns>
50     [MethodImpl(MethodImplOptions.AggressiveInlining)]
51     public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links,
↪     FormatOptions options)
52     {
53         if (links == null || links.Count == 0)
54         {
55             return string.Empty;
56         }
57
58         // Apply consecutive link grouping if enabled
59         var linksToFormat = options.GroupConsecutive ? GroupConsecutiveLinks(links) : links;
60
61         // Format each link with options
62         var formattedLinks = linksToFormat.Select(l => l.FormatWithOptions(options));
63         return string.Join(Environment.NewLine, formattedLinks);
64     }
65
66     /// <summary>
67     /// Group consecutive links with the same ID.
68     /// </summary>
69     private static List<Link<TLinkAddress>>
↪     GroupConsecutiveLinks<TLinkAddress>(IList<Link<TLinkAddress>> links)
70     {
71         if (links == null || links.Count == 0)
72         {
73             return new List<Link<TLinkAddress>>();
74         }
75
76         var grouped = new List<Link<TLinkAddress>>();
77         int i = 0;
78
79         while (i < links.Count)
80         {
81             var current = links[i];
82
83             // Look ahead for consecutive links with same ID
84             if (current.Id != null && current.Values != null && current.Values.Count > 0)
85             {
86                 // Collect all values with same ID
87                 var sameIdValues = new List<Link<TLinkAddress>>(current.Values);

```

```

88         int j = i + 1;
89
90         while (j < links.Count)
91         {
92             var nextLink = links[j];
93             if (EqualityComparer<TLinkAddress>.Default.Equals(nextLink.Id,
↪ current.Id) &&
94                 nextLink.Values != null && nextLink.Values.Count > 0)
95             {
96                 sameIdValues.AddRange(nextLink.Values);
97                 j++;
98             }
99             else
100             {
101                 break;
102             }
103         }
104
105         // If we found consecutive links, create grouped link
106         if (j > i + 1)
107         {
108             var groupedLink = new Link<TLinkAddress>(current.Id, sameIdValues);
109             grouped.Add(groupedLink);
110             i = j;
111             continue;
112         }
113     }
114
115     grouped.Add(current);
116     i++;
117 }
118
119     return grouped;
120 }
121 }
122 }

```

## 1.7 ./Link.Foundation.Links.Notation/Link.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Runtime.CompilerServices;
4  using System.Text;
5  using Platform.Collections;
6  using Platform.Collections.Lists;
7
8  namespace Link.Foundation.Links.Notation
9  {
10     /// <summary>
11     /// Represents a link in the Links Notation with an optional identifier and nested values.
12     /// Links can represent simple references, complex nested structures, or relationships
↪ between references to links.
13     /// </summary>
14     /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
↪ any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
15     public struct Link<TLinkAddress>: IEquatable<Link<TLinkAddress>>
16     {
17         private static readonly EqualityComparer<TLinkAddress> EqualityComparerInstance =
↪ EqualityComparer<TLinkAddress>.Default;
18
19         /// <summary>
20         /// Gets the identifier or address of this link. Can be null for anonymous links.
21         /// </summary>
22         public readonly TLinkAddress? Id;
23
24         /// <summary>
25         /// Gets the collection of nested link values. Can be null or empty for simple reference
↪ links.
26         /// </summary>
27         public readonly IList<Link<TLinkAddress>>? Values;
28
29         /// <summary>
30         /// Initializes a new link with the specified identifier and values.
31         /// </summary>
32         /// <param name="id">The link identifier or address.</param>
33         /// <param name="values">The nested link values.</param>
34         [MethodImpl(MethodImplOptions.AggressiveInlining)]
35         public Link(TLinkAddress? id, IList<Link<TLinkAddress>>? values) => (Id, Values) = (id,
↪ values);
36
37         /// <summary>

```

```

38     /// Initializes a new anonymous link with the specified values.
39     /// </summary>
40     /// <param name="values">The nested link values.</param>
41     [MethodImpl(MethodImplOptions.AggressiveInlining)]
42     public Link(ICollection<Link<TLinkAddress>> values) : this(default!, values) { }
43
44     /// <summary>
45     /// Initializes a new anonymous link with the specified array of values.
46     /// </summary>
47     /// <param name="values">The nested link values as a parameter array.</param>
48     [MethodImpl(MethodImplOptions.AggressiveInlining)]
49     public Link(params Link<TLinkAddress>[] values) : this(default!, values) { }
50
51     /// <summary>
52     /// Initializes a new simple reference link with the specified identifier.
53     /// </summary>
54     /// <param name="id">The link identifier or address.</param>
55     [MethodImpl(MethodImplOptions.AggressiveInlining)]
56     public Link(TLinkAddress id) : this(id, default!) { }
57
58     /// <summary>
59     /// Returns the string representation of this link in Links Notation format.
60     /// </summary>
61     /// <returns>A string representation of the link with proper escaping and
62     ↪ formatting.</returns>
63     [MethodImpl(MethodImplOptions.AggressiveInlining)]
64     public override string ToString() => Id == null ?
65         $"({GetValuesString()})" :
66         (Values == null || Values.Count == 0) ?
67             $"({EscapeReference(Id.ToString())})" :
68             $"({EscapeReference(Id.ToString())}: {GetValuesString()})";
69
70     /// <summary>
71     /// Gets the string representation of the link's values.
72     /// </summary>
73     /// <returns>A space-separated string of the values, or empty string if no values
74     ↪ exist.</returns>
75     [MethodImpl(MethodImplOptions.AggressiveInlining)]
76     public string GetValuesString()
77     {
78         if (Values == null || Values.Count == 0)
79         {
80             return "";
81         }
82         var sb = new StringBuilder();
83         for (int i = 0; i < Values.Count; i++)
84         {
85             if (i > 0)
86             {
87                 sb.Append(' ');
88             }
89             sb.Append(GetValueString(Values[i]));
90         }
91         return sb.ToString();
92     }
93
94     /// <summary>
95     /// Simplifies the link structure by unwrapping single-value containers and recursively
96     ↪ simplifying nested values.
97     /// </summary>
98     /// <returns>A simplified version of this link.</returns>
99     [MethodImpl(MethodImplOptions.AggressiveInlining)]
100     public Link<TLinkAddress> Simplify()
101     {
102         if (Values == null || Values.Count == 0)
103         {
104             return this;
105         }
106         else if (Values.Count == 1)
107         {
108             return Values[0];
109         }
110         else
111         {
112             var newValues = new Link<TLinkAddress>[Values.Count];
113             for (int i = 0; i < Values.Count; i++)
114             {
115                 newValues[i] = Values[i].Simplify();
116             }
117             return new Link<TLinkAddress>(newValues);
118         }
119     }

```

```

113     }
114     return new Link<TLinkAddress>(Id, newValues);
115 }
116 }
117
118 /// <summary>
119 /// Combines this link with another link to create a new link containing both as values.
120 /// </summary>
121 /// <param name="other">The other link to combine with.</param>
122 /// <returns>A new link containing both links as values.</returns>
123 [MethodImpl(MethodImplOptions.AggressiveInlining)]
124 public Link<TLinkAddress> Combine(Link<TLinkAddress> other) => new
↪ Link<TLinkAddress>(this, other);
125
126 /// <summary>
127 /// Gets the string representation of a link value, choosing the appropriate format.
128 /// </summary>
129 /// <param name="value">The link value to convert to string.</param>
130 /// <returns>The string representation of the link value.</returns>
131 [MethodImpl(MethodImplOptions.AggressiveInlining)]
132 public static string GetValueString(Link<TLinkAddress> value) =>
↪ value.ToLinkOrIdString();
133
134 /// <summary>
135 ↪ /// Escapes a reference string for safe use in Links Notation format by adding quotes if
136 ↪ necessary.
137 /// </summary>
138 /// <param name="reference">The reference string to escape.</param>
139 /// <returns>The escaped reference string with appropriate quoting.</returns>
140 public static string EscapeReference(string? reference)
141 {
142     if (reference == null)
143     {
144         return "";
145     }
146     // The empty reference is written as a bare delimiter pair, so that it reads
147     // back as itself instead of disappearing from the document.
148     if (reference.Length == 0)
149     {
150         return "\"\"";
151     }
152     // A reference that begins with a "#" has to be quoted, or it would read
153     // back as a comment. A "#" anywhere else in a reference is content
154     // ("issue#1047"), so only the first character matters.
155     if (
156         reference.StartsWith("#") ||
157         reference.Contains(":") ||
158         reference.Contains("(") ||
159         reference.Contains(")") ||
160         reference.Contains(" ") ||
161         reference.Contains("\t") ||
162         reference.Contains("\n") ||
163         reference.Contains("\r") ||
164         reference.Contains("\\"))
165     {
166         return $"{reference}";
167     }
168     else if (reference.Contains("\""))
169     {
170         return $"\"{reference}\"";
171     }
172     else
173     {
174         return reference;
175     }
176 }
177
178 /// <summary>
179 ↪ /// Converts the link to its string representation, choosing between simple reference
180 ↪ format or full link format.
181 /// </summary>
182 /// <returns>A string representation optimized for the link's content.</returns>
183 [MethodImpl(MethodImplOptions.AggressiveInlining)]
184 public string ToLinkOrIdString() => (Values == null || Values.Count == 0) ? (Id == null
↪ ? "" : EscapeReference(Id?.ToString())) : ToString();
185
186 /// <summary>

```

```

186     /// Implicitly converts a value to a simple reference link.
187     /// </summary>
188     /// <param name="value">The value to convert.</param>
189     [MethodImpl(MethodImplOptions.AggressiveInlining)]
190     public static implicit operator Link<TLinkAddress>(TLinkAddress value) => new
↳ Link<TLinkAddress>(value);

191     /// <summary>
192     /// Implicitly converts a tuple of identifier and values to a link.
193     /// </summary>
194     /// <param name="value">The tuple containing identifier and values.</param>
195     [MethodImpl(MethodImplOptions.AggressiveInlining)]
196     public static implicit operator Link<TLinkAddress>((TLinkAddress,
↳ IList<Link<TLinkAddress>>) value) => new(value.Item1, value.Item2);

198     /// <summary>
199     /// Implicitly converts a source-target pair to an anonymous link.
200     /// </summary>
201     /// <param name="value">The tuple containing source and target links.</param>
202     [MethodImpl(MethodImplOptions.AggressiveInlining)]
203     public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source,
↳ Link<TLinkAddress> target) value) => new(value.source, value.target);

205     /// <summary>
206     /// Implicitly converts a tuple of identifier, source, and target to a link.
207     /// </summary>
208     /// <param name="value">The tuple containing id, source and target.</param>
209     [MethodImpl(MethodImplOptions.AggressiveInlining)]
210     public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id,
↳ Link<TLinkAddress> source, Link<TLinkAddress> target) value) => new(value.id.Id, new[] {
↳ value.source, value.target });

212     /// <summary>
213     /// Implicitly converts a source-linker-target triplet to an anonymous link.
214     /// </summary>
215     /// <param name="value">The tuple containing source, linker, and target links.</param>
216     [MethodImpl(MethodImplOptions.AggressiveInlining)]
217     public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source,
↳ Link<TLinkAddress> linker, Link<TLinkAddress> target) value) => new(value.source,
↳ value.linker, value.target);

219     /// <summary>
220     /// Implicitly converts a tuple of identifier, source, linker, and target to a link.
221     /// </summary>
222     /// <param name="value">The tuple containing id, source, linker, and target.</param>
223     [MethodImpl(MethodImplOptions.AggressiveInlining)]
224     public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id,
↳ Link<TLinkAddress> source, Link<TLinkAddress> linker, Link<TLinkAddress> target) value) =>
↳ new(value.id.Id, new[] { value.source, value.linker, value.target });

226     /// <summary>
227     /// Determines whether the specified object is equal to this link.
228     /// </summary>
229     /// <param name="obj">The object to compare with this link.</param>
230     /// <returns>True if the specified object is equal to this link; otherwise,
↳ false.</returns>
232     [MethodImpl(MethodImplOptions.AggressiveInlining)]
233     public override bool Equals(object? obj) => obj is Link<TLinkAddress> link &&
↳ Equals(link);

234     /// <summary>
235     /// Returns the hash code for this link.
236     /// </summary>
237     /// <returns>A hash code for this link.</returns>
238     [MethodImpl(MethodImplOptions.AggressiveInlining)]
239     public override int GetHashCode() => (Id, (Values ??
↳ Array.Empty<Link<TLinkAddress>>()).GenerateHashCode()).GetHashCode();

241     /// <summary>
242     /// Indicates whether the current link is equal to another link.
243     /// Two anonymous links (both with null Ids) are considered equal if their values are
↳ equal.
245     /// </summary>
246     /// <param name="other">The link to compare with this link.</param>
247     /// <returns>True if the current link is equal to the other parameter; otherwise,
↳ false.</returns>
248     [MethodImpl(MethodImplOptions.AggressiveInlining)]

```

```

249     public bool Equals(Link<TLinkAddress> other)
250     {
251         // Both have null IDs - compare only values
252         if (Id == null && other.Id == null)
253         {
254             return (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ??
↪ Array.Empty<Link<TLinkAddress>>());
255         }
256
257         // Only one has null ID - not equal
258         if (Id == null || other.Id == null)
259         {
260             return false;
261         }
262
263         // Both have IDs - compare IDs and values
264         return EqualityComparerInstance.Equals(Id, other.Id) &&
265             (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ??
↪ Array.Empty<Link<TLinkAddress>>());
266     }
267
268     /// <summary>
269     /// Determines whether two link instances are equal.
270     /// </summary>
271     /// <param name="left">The first link to compare.</param>
272     /// <param name="right">The second link to compare.</param>
273     /// <returns>True if the links are equal; otherwise, false.</returns>
274     [MethodImpl(MethodImplOptions.AggressiveInlining)]
275     public static bool operator ==(Link<TLinkAddress> left, Link<TLinkAddress> right) =>
↪ left.Equals(right);
276
277     /// <summary>
278     /// Determines whether two link instances are not equal.
279     /// </summary>
280     /// <param name="left">The first link to compare.</param>
281     /// <param name="right">The second link to compare.</param>
282     /// <returns>True if the links are not equal; otherwise, false.</returns>
283     [MethodImpl(MethodImplOptions.AggressiveInlining)]
284     public static bool operator !=(Link<TLinkAddress> left, Link<TLinkAddress> right) =>
↪ !(left == right);
285 }
286 }

```

## 1.8 ./Link.Foundation.Links.Notation/LinkFormatExtensions.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Runtime.CompilerServices;
5  using System.Text;
6
7  namespace Link.Foundation.Links.Notation
8  {
9      /// <summary>
10     /// Provides extension methods for formatting <see cref="Link{TLinkAddress}"/> instances
↪ with FormatOptions.
11     /// </summary>
12     public static class LinkFormatExtensions
13     {
14         /// <summary>
15         /// Format a link using FormatOptions configuration.
16         /// </summary>
17         /// <typeparam name="TLinkAddress">The type used for link
↪ addresses/identifiers.</typeparam>
18         /// <param name="link">The link to format.</param>
19         /// <param name="options">The FormatOptions to use.</param>
20         /// <returns>Formatted string representation.</returns>
21         public static string FormatWithOptions<TLinkAddress>(this Link<TLinkAddress> link,
↪ FormatOptions options)
22         {
23             // Empty link
24             if (link.Id == null && (link.Values == null || link.Values.Count == 0))
25             {
26                 return options.LessParentheses ? "" : "()";
27             }
28
29             // Link with only ID, no values
30             if (link.Values == null || link.Values.Count == 0)
31             {

```

```

32     var escapedId = Link<TLinkAddress>.EscapeReference(link.Id?.ToString());
33     return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
34         escapedId :
35         $"({escapedId})";
36 }
37
38 // Check if we should use indented format
39 bool shouldIndent = false;
40 if (options.ShouldIndentByRefCount(link.Values.Count))
41 {
42     shouldIndent = true;
43 }
44 else
45 {
46     // Try inline format first
47     var valuesStr = string.Join(" ", link.Values.Select(v => GetValueString(v)));
48     string testLine;
49     if (link.Id != null)
50     {
51         var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
52         testLine = options.LessParentheses ? $"{idStr}: {valuesStr}" : $"{idStr}:
↪ {valuesStr}";
53     }
54     else
55     {
56         testLine = options.LessParentheses ? valuesStr : $"({valuesStr})";
57     }
58
59     if (options.ShouldIndentByLength(testLine))
60     {
61         shouldIndent = true;
62     }
63 }
64
65 // Format with indentation if needed
66 if (shouldIndent && options.PreferInline == false)
67 {
68     return FormatIndented(link, options);
69 }
70
71 // Standard inline formatting
72 var values = string.Join(" ", link.Values.Select(v => GetValueString(v)));
73
74 // Link with values only (null id)
75 if (link.Id == null)
76 {
77     if (options.LessParentheses)
78     {
79         var allSimple = link.Values.All(v => v.Values == null || v.Values.Count ==
↪ 0);
80         if (allSimple)
81         {
82             return string.Join(" ", link.Values.Select(v =>
↪ Link<TLinkAddress>.EscapeReference(v.Id?.ToString())));
83         }
84         return values;
85     }
86     return $"({values})";
87 }
88
89 // Link with ID and values
90 var id = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
91 var withColon = $"{id}: {values}";
92 return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
93     withColon :
94     $"({withColon})";
95 }
96
97 /// <summary>
98 /// Format a link with indentation.
99 /// </summary>
100 private static string FormatIndented<TLinkAddress>(Link<TLinkAddress> link,
↪ FormatOptions options)
101 {
102     // FormatIndented is only reached with a non-empty Values collection,
103     // but the nullable annotation on Link<T>.Values does not know that.
104     var values = link.Values ?? Array.Empty<Link<TLinkAddress>>();
105
106     if (link.Id == null)

```

```

107     {
108         // Values only - format each on separate line
109         var lines = values.Select(v => options.IndentString + GetValueString(v));
110         return string.Join(Environment.NewLine, lines);
111     }
112
113     // Link with ID - format as id:\n value1\n value2
114     var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
115     var sb = new StringBuilder();
116     sb.Append($"{{idStr}}:");
117
118     foreach (var v in values)
119     {
120         sb.Append(Environment.NewLine);
121         sb.Append(options.IndentString);
122         sb.Append(GetValueString(v));
123     }
124
125     return sb.ToString();
126 }
127
128 /// <summary>
129 /// Get the string representation of a value.
130 /// </summary>
131 private static string GetValueString<TLinkAddress>(Link<TLinkAddress> value)
132 {
133     return value.ToLinkOrIdString();
134 }
135
136 /// <summary>
137 /// Check if a string needs to be wrapped in parentheses.
138 /// </summary>
139 private static bool NeedsParentheses(string? s)
140 {
141     return s != null && (s.Contains(" ") || s.Contains(":") || s.Contains("(") ||
↪ s.Contains(")"));
142 }
143 }
144 }

```

## 1.9 ./Link.Foundation.Links.Notation/LinksGroup.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.Runtime.CompilerServices;
4 using Platform.Collections;
5 using Platform.Collections.Lists;
6
7 namespace Link.Foundation.Links.Notation
8 {
9     /// <summary>
10     /// Represents a group of links with hierarchical structure, where each group contains a
↪ primary link and optional nested groups.
11     /// This structure supports the indentation-based syntax of the Links Notation.
12     /// </summary>
13     /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
↪ any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
14     public struct LinksGroup<TLinkAddress> : IEquatable<LinksGroup<TLinkAddress>>
15     {
16         /// <summary>
17         /// Gets or sets the primary link of this group.
18         /// </summary>
19         public Link<TLinkAddress> Link
20         {
21             [MethodImpl(MethodImplOptions.AggressiveInlining)]
22             get;
23             [MethodImpl(MethodImplOptions.AggressiveInlining)]
24             set;
25         }
26
27         /// <summary>
28         /// Gets or sets the collection of nested groups within this group. Can be null if no
↪ nested groups exist.
29         /// </summary>
30         public IList<LinksGroup<TLinkAddress>>? Groups
31         {
32             [MethodImpl(MethodImplOptions.AggressiveInlining)]
33             get;
34             [MethodImpl(MethodImplOptions.AggressiveInlining)]
35             set;

```

```

36     }
37
38     /// <summary>
39     /// Gets or sets a value indicating whether this group was written as a parenthesized
↳ group.
40     /// A parenthesized group is a link in its own right, so it keeps its identifier and
↳ values
41     /// instead of being unwrapped when it is used as a child of the indented identifier
↳ syntax.
42     /// This is a parse-time hint and therefore takes no part in equality comparisons.
43     /// </summary>
44     internal bool IsParenthesized
45     {
46         [MethodImpl(MethodImplOptions.AggressiveInlining)]
47         get;
48         [MethodImpl(MethodImplOptions.AggressiveInlining)]
49         set;
50     }
51
52     /// <summary>
53     /// Initializes a new links group with the specified link and nested groups.
54     /// </summary>
55     /// <param name="link">The primary link of this group.</param>
56     /// <param name="groups">The nested groups within this group.</param>
57     [MethodImpl(MethodImplOptions.AggressiveInlining)]
58     public LinksGroup(Link<TLinkAddress> link, IList<LinksGroup<TLinkAddress>>? groups)
59     {
60         Link = link;
61         Groups = groups;
62     }
63
64     /// <summary>
65     /// Initializes a new links group with the specified link and no nested groups.
66     /// </summary>
67     /// <param name="link">The primary link of this group.</param>
68     [MethodImpl(MethodImplOptions.AggressiveInlining)]
69     public LinksGroup(Link<TLinkAddress> link) : this(link, null) { }
70
71     /// <summary>
72     /// Implicitly converts a links group to a list of links by flattening the hierarchical
↳ structure.
73     /// </summary>
74     /// <param name="value">The links group to convert.</param>
75     [MethodImpl(MethodImplOptions.AggressiveInlining)]
76     public static implicit operator List<Link<TLinkAddress>>(LinksGroup<TLinkAddress> value)
↳ => value.ToLinksList();
77
78     /// <summary>
79     /// Converts this links group to a flat list of links, preserving the hierarchical
↳ relationships.
80     /// </summary>
81     /// <returns>A list of links representing the flattened structure of this
↳ group.</returns>
82     [MethodImpl(MethodImplOptions.AggressiveInlining)]
83     public List<Link<TLinkAddress>> ToLinksList()
84     {
85         var list = new List<Link<TLinkAddress>>();
86         AppendToLinksList(list);
87         return list;
88     }
89
90     /// <summary>
91     /// Appends this links group to the specified list as flattened links.
92     /// </summary>
93     /// <param name="list">The list to append the links to.</param>
94     [MethodImpl(MethodImplOptions.AggressiveInlining)]
95     public void AppendToLinksList(List<Link<TLinkAddress>> list) => AppendToLinksList(list,
↳ Link, this);
96
97     /// <summary>
98     /// Recursively appends a links group to a list, combining dependencies with nested
↳ groups to create a flattened structure.
99     /// </summary>
100    /// <param name="list">The list to append the links to.</param>
101    /// <param name="dependency">The dependency link to combine with nested groups.</param>
102    /// <param name="group">The links group to process.</param>
103    [MethodImpl(MethodImplOptions.AggressiveInlining)]
104    public static void AppendToLinksList(List<Link<TLinkAddress>> list, Link<TLinkAddress>
↳ dependency, LinksGroup<TLinkAddress> group)

```

```

105     {
106         list.Add(dependency);
107         var groups = group.Groups;
108         if (groups != null && !groups.IsNullOrEmpty())
109         {
110             for (int i = 0; i < groups.Count; i++)
111             {
112                 var innerGroup = groups[i];
113                 AppendToLinksList(list, dependency.Combine(innerGroup.Link), innerGroup);
114             }
115         }
116     }
117
118     /// <summary>
119     /// Determines whether the specified object is equal to this links group.
120     /// </summary>
121     /// <param name="obj">The object to compare with this links group.</param>
122     /// <returns>True if the specified object is equal to this links group; otherwise,
123     ↪ false.</returns>
124     [MethodImpl(MethodImplOptions.AggressiveInlining)]
125     public override bool Equals(object? obj) => obj is LinksGroup<TLinkAddress> linksGroup ?
126     ↪ Equals(linksGroup) : false;
127
128     /// <summary>
129     /// Returns the hash code for this links group.
130     /// </summary>
131     /// <returns>A hash code for this links group.</returns>
132     [MethodImpl(MethodImplOptions.AggressiveInlining)]
133     public override int GetHashCode() => (Link, (Groups ??
134     ↪ Array.Empty<LinksGroup<TLinkAddress>>()).GenerateHashCode()).GetHashCode();
135
136     /// <summary>
137     /// Indicates whether the current links group is equal to another links group.
138     /// </summary>
139     /// <param name="other">The links group to compare with this links group.</param>
140     /// <returns>True if the current links group is equal to the other parameter; otherwise,
141     ↪ false.</returns>
142     [MethodImpl(MethodImplOptions.AggressiveInlining)]
143     public bool Equals(LinksGroup<TLinkAddress> other) => Link.Equals(other.Link) && (Groups
144     ↪ ?? Array.Empty<LinksGroup<TLinkAddress>>()).EqualTo(other.Groups ??
145     ↪ Array.Empty<LinksGroup<TLinkAddress>>());
146
147     /// <summary>
148     /// Determines whether two links group instances are equal.
149     /// </summary>
150     /// <param name="left">The first links group to compare.</param>
151     /// <param name="right">The second links group to compare.</param>
152     /// <returns>True if the links groups are equal; otherwise, false.</returns>
153     [MethodImpl(MethodImplOptions.AggressiveInlining)]
154     public static bool operator ==(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
155     ↪ right) => left.Equals(right);
156
157     /// <summary>
158     /// Determines whether two links group instances are not equal.
159     /// </summary>
160     /// <param name="left">The first links group to compare.</param>
161     /// <param name="right">The second links group to compare.</param>
162     /// <returns>True if the links groups are not equal; otherwise, false.</returns>
163     [MethodImpl(MethodImplOptions.AggressiveInlining)]
164     public static bool operator !=(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
165     ↪ right) => !(left == right);
166
167     /// <summary>
168     /// Returns a string representation of this links group.
169     /// </summary>
170     /// <returns>A string representation of this links group.</returns>
171     [MethodImpl(MethodImplOptions.AggressiveInlining)]
172     public override string ToString() => ToLinksList().Format();
173 }

```

## 1.10 ./Link.Foundation.Links.Notation/ParseException.cs

```

1 using System;
2 using System.Text;
3
4 namespace Link.Foundation.Links.Notation
5 {
6     /// <summary>

```

```

7  /// The error raised when a document does not parse, carrying the position the parser
8  /// stopped at.
9  /// </summary>
10 /// <remarks>
11 /// The position is the furthest one the parser reached, which is the character the
12 /// document stops making sense at rather than the point the last alternative gave up
13 /// on. It derives from <see cref="FormatException"/>, the exception the generated
14 /// parser raised on its own, so code that catches that keeps working.
15 /// A document nested deeper than <see cref="Parser.MaxDepth"/> is refused with this
16 /// exception too; then <see cref="MaxDepth"/> says how deep the nesting may go, and the
17 /// position is the group or the line that is one level too deep.
18 /// </remarks>
19 public class ParseException : FormatException
20 {
21     /// <summary>The number of characters a quoted line is cut down to.</summary>
22     private const int QuotedLineWidth = 80;
23
24     /// <summary>What a message writes in place of the part of a long line it left
25     out.</summary>
26     private const string Ellipsis = "...";
27
28     /// <summary>
29     /// Creates an exception describing where <paramref name="subject"/> stopped parsing.
30     /// </summary>
31     /// <param name="subject">The document that failed to parse.</param>
32     /// <param name="offset">Offset of the position the parser stopped at.</param>
33     /// <param name="innerException">The error the generated parser raised, if any.</param>
34     public ParseException(string subject, int offset, Exception? innerException = null)
35         : this(Locate(subject ?? string.Empty, offset), null, innerException)
36     {
37     }
38
39     /// <summary>
40     /// Creates an exception saying that <paramref name="subject"/> nests deeper than
41     /// <paramref name="maxDepth"/> at <paramref name="offset"/>.
42     /// </summary>
43     /// <param name="subject">The document that nests too deeply.</param>
44     /// <param name="offset">Offset of the group or the line that is one level too
45     deep.</param>
46     /// <param name="maxDepth">The deepest nesting allowed.</param>
47     /// <param name="innerException">The error the parser raised, if any.</param>
48     public ParseException(string subject, int offset, int maxDepth, Exception?
49     innerException = null)
50         : this(Locate(subject ?? string.Empty, offset), maxDepth, innerException)
51     {
52     }
53
54     private ParseException(Position position, int? maxDepth, Exception? innerException)
55         : base(Describe(position, maxDepth), innerException)
56     {
57         Offset = position.Offset;
58         Line = position.Line;
59         Column = position.Column;
60         Found = position.Found;
61         LineText = position.LineText;
62         MaxDepth = maxDepth;
63     }
64
65     /// <summary>Offset of the offending position from the start of the document.</summary>
66     public int Offset { get; }
67
68     /// <summary>Line the offending position is on, counted from 1.</summary>
69     public int Line { get; }
70
71     /// <summary>Column the offending position is at, counted from 1.</summary>
72     public int Column { get; }
73
74     /// <summary>The character found instead, or null at the end of the document.</summary>
75     public char? Found { get; }
76
77     /// <summary>The offending line, as written, without its line ending.</summary>
78     public string LineText { get; }
79
80     /// <summary>
81     /// The deepest nesting allowed, when the document is nested deeper than that;
82     /// <see langword="null"/> for any other error.
83     /// </summary>
84     public int? MaxDepth { get; }

```

```

83     /// <summary>
84     /// The one-line summary: where the parser stopped and what stands there, or that the
85     /// nesting is too deep there.
86     /// </summary>
87     public string Summary => Summarize(new Position(Offset, Line, Column, Found, LineText),
↪ MaxDepth);

88     /// <summary>
89     /// The offending line with a caret under the offending column, quoted the way a
90     /// compiler quotes source. A long line is shown as a window around the caret, so the
91     /// message stays the same size whether the document has ten lines or fifteen hundred.
92     /// </summary>
93     public string Snippet => Quote(new Position(Offset, Line, Column, Found, LineText));

94     /// <summary>Where the parser stopped, and what the document holds there.</summary>
95     private readonly record struct Position(
96         int Offset,
97         int Line,
98         int Column,
99         char? Found,
100         string LineText);

101     /// <summary>The message the exception carries: the summary and the quoted
102     ↪ line.</summary>
103     private static string Describe(Position position, int? maxDepth) =>
104         $"{(maxDepth.HasValue ? "Nesting too deep" : "Syntax error")} at
105         ↪ {Summarize(position, maxDepth)}\n{Quote(position)}";

106     private static string Summarize(Position position, int? maxDepth)
107     {
108         if (maxDepth.HasValue)
109         {
110             return $"line {position.Line}, column {position.Column}: nesting depth exceeds
111             ↪ the maximum of {maxDepth.Value}";
112         }
113         var found = position.Found.HasValue
114             ? $"{Escape(position.Found.Value)}\n"
115             : "end of input";
116         return $"line {position.Line}, column {position.Column}: unexpected {found}";
117     }

118     private static string Quote(Position position)
119     {
120         var (quoted, column) = QuoteLine(position.LineText, position.Column);
121         var number = position.Line.ToString();
122         var gutter = new string(' ', number.Length);
123         return $"{number} | {quoted}\n{gutter} | {new string(' ', column - 1)}~";
124     }

125     private static string Escape(char character) => character switch
126     {
127         '\n' => "\\n",
128         '\r' => "\\r",
129         '\t' => "\\t",
130         '"' => "\\\"",
131         '\\' => "\\\\",
132         => character.ToString(),
133     };

134     /// <summary>
135     /// Turns the position the parser stopped at into a line, a column and the line
136     /// itself, so the message can point at the defect instead of quoting the rest of the
137     /// document.
138     /// </summary>
139     private static Position Locate(string document, int offset)
140     {
141         offset = Math.Max(0, Math.Min(offset, document.Length));
142         var line = 1;
143         var lineStart = 0;
144         for (var index = 0; index < offset; index++)
145         {
146             if (document[index] == '\n')
147             {
148                 line++;
149                 lineStart = index + 1;
150             }
151         }
152     }
153
154
155
156
157

```

```

158     var lineEnd = document.IndexOf('\n', lineStart);
159     if (lineEnd < 0) lineEnd = document.Length;
160
161     var lineText = document.Substring(lineStart, lineEnd - lineStart).TrimEnd('\r');
162     var found = offset < document.Length ? document[offset] : (char?)null;
163     return new Position(offset, line, offset - lineStart + 1, found, lineText);
164 }
165
166 /// <summary>
167 /// Cuts <paramref name="line"/> down to a window around <paramref name="column"/>, and
168 /// says which column the offending character sits at in that window. Both columns
169 /// count from 1.
170 /// </summary>
171 private static (string Quoted, int Column) QuoteLine(string line, int column)
172 {
173     if (line.Length <= QuotedLineWidth) return (line, column);
174
175     var target = column - 1;
176     var lastStart = line.Length - QuotedLineWidth;
177     var start = Math.Min(Math.Max(target - (QuotedLineWidth / 2), 0), lastStart);
178     var end = start + QuotedLineWidth;
179
180     var quoted = new StringBuilder();
181     if (start > 0) quoted.Append(Ellipsis);
182     quoted.Append(line, start, QuotedLineWidth);
183     if (end < line.Length) quoted.Append(Ellipsis);
184
185     var shift = start > 0 ? Ellipsis.Length : 0;
186     return (quoted.ToString(), target - start + shift + 1);
187 }
188 }
189 }

```

## 1.11 ./Link.Foundation.Links.Notation/Parser.cs

```

1 using System;
2 using System.Collections.Generic;
3
4 namespace Link.Foundation.Links.Notation
5 {
6     /// <summary>
7     /// Parses Links Notation documents into links.
8     /// </summary>
9     /// <remarks>
10    /// The rules live in <c>Parser.peg</c> and are compiled into
11    /// <see cref="GeneratedParser"/>; this class runs them and, when a document does not
12    /// parse, reports the position the parser stopped at instead of the bare
13    /// <see cref="FormatException"/> the generated parser raises.
14    /// </remarks>
15    public class Parser
16    {
17        /// <summary>
18        /// How deep links may nest when nothing says otherwise.
19        /// </summary>
20        /// <remarks>
21        /// The parser recurses once per level of nesting, and running out of stack ends the
22        /// process where no caller can catch it. The limit is set well below what fits in a
23        /// 1 MiB thread stack, and it is the same in every Links Notation implementation, so
24        /// a document one of them reads is not too deep for another.
25        /// </remarks>
26        public const int DefaultMaxDepth = 64;
27
28        /// <summary>
29        /// Creates a parser.
30        /// </summary>
31        /// <param name="comments">
32        /// When <see langword="false"/>, <c>#</c> is read as an ordinary character
33        /// instead of the start of a comment.
34        /// </param>
35        public Parser(bool comments = true) : this(comments, DefaultMaxDepth)
36        {
37        }
38
39        /// <summary>
40        /// Creates a parser that refuses links nested deeper than <paramref name="maxDepth"/>.
41        /// </summary>
42        /// <param name="comments">
43        /// When <see langword="false"/>, <c>#</c> is read as an ordinary character
44        /// instead of the start of a comment.
45        /// </param>

```

```

46     /// <param name="maxDepth">How deep links may nest; see <see cref="MaxDepth"/>.</param>
47     /// <exception cref="ArgumentOutOfRangeException"><paramref name="maxDepth"/> is
    ↪ negative.</exception>
48     public Parser(bool comments, int maxDepth)
49     {
50         Comments = comments;
51         MaxDepth = maxDepth;
52     }
53
54     /// <summary>
55     /// Whether <c>#</c> starts a comment that runs to the end of its line.
56     /// </summary>
57     public bool Comments { get; }
58
59     /// <summary>
60     /// How deep links may nest. Every parenthesized group and every indentation level
61     /// is one level, and the lines of a document are at level 0, so with a limit of 1
62     /// <c>(a)</c> parses while <c>((a))</c>, <c>(a (b))</c> and a group on an indented
63     /// line do not. A document nested deeper is refused with a
64     /// <see cref="ParseException"/> whose <see cref="ParseException.MaxDepth"/> is set,
65     /// rather than recursed into until the stack runs out.
66     /// </summary>
67     /// <exception cref="ArgumentOutOfRangeException">The value is negative.</exception>
68     public int MaxDepth
69     {
70         get => maxDepth;
71         init => maxDepth = value >= 0
72             ? value
73             : throw new ArgumentOutOfRangeException(nameof(MaxDepth), value, "Maximum
    ↪ nesting depth cannot be negative.");
74     }
75
76     private readonly int maxDepth;
77
78     /// <summary>
79     /// Parses a Links Notation document.
80     /// </summary>
81     /// <param name="subject">The document to parse.</param>
82     /// <param name="fileName">The optional file name to use in error messages.</param>
83     /// <returns>The links the document holds.</returns>
84     /// <exception cref="ParseException">
85     /// Thrown when the document does not parse, or nests deeper than
86     /// <see cref="MaxDepth"/>. The exception carries the line, the column and the
87     /// offending line of the position the parser stopped at.
88     /// </exception>
89     public IList<Link<string>> Parse(string subject, string? fileName = null)
90     {
91         // Comments are blanked rather than removed, so a position reported for the
92         // prepared document is the same position in the document the caller wrote.
93         var prepared = Comments ? Notation.Comments.StripComments(subject) : subject;
94         var tracer = new FurthestFailureTracer();
95         var parser = new GeneratedParser { Tracer = tracer, MaxDepth = MaxDepth };
96         try
97         {
98             return parser.Parse(prepared, fileName);
99         }
100         catch (GeneratedParser.NestingTooDeepException error)
101         {
102             throw new ParseException(subject, error.Offset, error.MaxDepth, error);
103         }
104         catch (FormatException error) when (error is not ParseException)
105         {
106             throw new ParseException(subject, tracer.Furthest, error);
107         }
108     }
109 }
110 }

```

## 1.12 ./Link.Foundation.Links.Notation/Quotes.cs

```

1 using System.Text;
2
3 namespace Link.Foundation.Links.Notation
4 {
5     /// <summary>
6     /// Reads a delimited reference, the shape shared by the grammar and by
7     /// <see cref="Comments"/>.
8     /// </summary>
9     /// <remarks>
10    /// Any number N of delimiters opens and closes a reference, and 2N delimiters stand

```

```

11  /// for N written as content. The grammar reads references through this class, so
12  /// blanking comments reaches a reference exactly as far as the parser does.
13  /// </remarks>
14  internal static class Quotes
15  {
16      /// <summary>
17      /// Reports whether the character is one of the delimiters a reference can be
18      /// written between.
19      /// </summary>
20      /// <param name="character">The character to test.</param>
21      /// <returns><see langword="true"/> when the character opens a reference.</returns>
22      internal static bool IsQuote(char character) =>
23          character == '"' || character == '\'' || character == '`';
24
25      /// <summary>
26      /// A body written between an even run of delimiters is substantive when it
27      /// holds at least one visible character and does not straddle a parenthesis.
28      /// An even run can always be read as delimiter pairs enclosing nothing, so the
29      /// n-quote reading is only taken when it carries something the pairs cannot.
30      /// </summary>
31      /// <param name="content">The body to weigh.</param>
32      /// <returns><see langword="true"/> when the body carries something a pair
33  ↪ cannot.</returns>
34      internal static bool IsSubstantiveBody(string content)
35      {
36          int depth = 0;
37          bool hasVisible = false;
38
39          foreach (var c in content)
40          {
41              if (c == '(')
42              {
43                  depth++;
44              }
45              else if (c == ')')
46              {
47                  depth--;
48                  if (depth < 0) return false;
49              }
50
51              if (!char.IsWhiteSpace(c)) hasVisible = true;
52          }
53
54          return hasVisible && depth == 0;
55
56      /// <summary>
57      /// Reads the reference starting at the given position.
58      /// </summary>
59      /// <param name="subject">The whole document being read.</param>
60      /// <param name="start">The position of the opening delimiter.</param>
61      /// <param name="value">The content of the reference, without its delimiters.</param>
62      /// <param name="length">How many characters the reference spans.</param>
63      /// <returns><see langword="true"/> when a reference was read at that position.</returns>
64      internal static bool TryParse(string? subject, int start, out string? value, out int
65  ↪ length)
66      {
67          value = null;
68          length = 0;
69          if (subject == null || start >= subject.Length || !IsQuote(subject[start])) return
69  ↪ false;
70
71          char quoteChar = subject[start];
72
73          // Count opening quotes
74          int quoteCount = 0;
75          int pos = start;
76          while (pos < subject.Length && subject[pos] == quoteChar)
77          {
78              quoteCount++;
79              pos++;
80          }
81
82          bool isEvenRun = quoteCount % 2 == 0;
83
84          string openClose = new string(quoteChar, quoteCount);
85          string escapeSeq = new string(quoteChar, quoteCount * 2);
86
87          var content = new StringBuilder();

```

```

87 while (pos < subject.Length)
88 {
89     // Check for escape sequence (2*N quotes)
90     if (pos + escapeSeq.Length <= subject.Length &&
91         string.CompareOrdinal(subject, pos, escapeSeq, 0, escapeSeq.Length) == 0)
92     {
93         content.Append(openClose); // 2*N quotes become N quotes
94         pos += escapeSeq.Length;
95         continue;
96     }
97
98     // Check for closing quotes (exactly N quotes, not more)
99     if (pos + quoteCount <= subject.Length &&
100        string.CompareOrdinal(subject, pos, openClose, 0, quoteCount) == 0)
101     {
102         int afterClose = pos + quoteCount;
103         if (afterClose >= subject.Length || subject[afterClose] != quoteChar)
104         {
105             // Found valid closing
106             if (isEvenRun && !IsSubstantiveBody(content.ToString()))
107             {
108                 return SetEmptyReference(isEvenRun, quoteCount, out value, out
↪ length);
109             }
110
111             value = content.ToString();
112             length = afterClose - start;
113             return true;
114         }
115     }
116
117     // Take next character
118     content.Append(subject[pos]);
119     pos++;
120 }
121
122 // No valid closing found
123 return SetEmptyReference(isEvenRun, quoteCount, out value, out length);
124 }
125
126 /// <summary>
127 /// Finds where the reference starting at the given position ends.
128 /// </summary>
129 /// <param name="subject">The whole document being read.</param>
130 /// <param name="start">The position of the opening delimiter.</param>
131 /// <returns>
132 /// The position right after the closing delimiters, or -1 when no reference
133 /// starts there.
134 /// </returns>
135 internal static int ReferenceEnd(string subject, int start) =>
136     TryParse(subject, start, out var ignored, out var length) ? start + length : -1;
137
138 /// <summary>
139 /// A run of an even number of delimiters that does not open a reference with a
140 /// substantive body is the empty reference: the shortest reading, a bare
141 /// delimiter pair enclosing nothing, wins over a longer n-quote delimiter.
142 /// </summary>
143 private static bool SetEmptyReference(bool isEvenRun, int quoteCount, out string? value,
↪ out int length)
144 {
145     if (!isEvenRun)
146     {
147         value = null;
148         length = 0;
149         return false;
150     }
151
152     value = string.Empty;
153     length = quoteCount;
154     return true;
155 }
156 }
157 }

```

### 1.13 ./Link.Foundation.Links.Notation/StreamParseException.cs

```

1 using System;
2
3 namespace Link.Foundation.Links.Notation
4 {

```

```

5  /// <summary>A parser error located relative to the complete input stream.</summary>
6  public sealed class StreamParseException : FormatException
7  {
8      internal StreamParseException(ParseException error, int offset, int line, int column)
9          : base($"Stream parse error at line {line}, column {column}: {error.Message}", error)
10     {
11         Offset = offset;
12         Line = line;
13         Column = column;
14         ParseError = error;
15     }
16
17     /// <summary>The canonical parser error raised for the buffered record.</summary>
18     public ParseException ParseError { get; }
19
20     /// <summary>Absolute character offset from the start of the stream.</summary>
21     public int Offset { get; }
22
23     /// <summary>One-based line number in the complete stream.</summary>
24     public int Line { get; }
25
26     /// <summary>One-based column number in the complete stream.</summary>
27     public int Column { get; }
28 }
29 }

```

#### 1.14 ./Link.Foundation.Links.Notation/StreamParser.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Runtime.CompilerServices;
4  using System.Text;
5  using System.Threading;
6  using System.Threading.Tasks;
7
8  namespace Link.Foundation.Links.Notation
9  {
10     /// <summary>Incrementally emits complete top-level Links Notation records.</summary>
11     public sealed class StreamParser
12     {
13         private const int DefaultMaxBufferSize = 10 * 1024 * 1024;
14         private const string BeforeReference = " \t\n\r(";
15         private const string BeforeComment = " \t\n\r";
16
17         private readonly Parser parser;
18         private readonly StringBuilder buffer = new();
19         private readonly StringBuilder currentLine = new();
20         private readonly List<Link<string>> links = new();
21         private int? baseIndentation;
22         private bool lineClassified;
23         private int offset;
24         private int line;
25         private int column;
26         private int segmentOffset;
27         private int segmentLine;
28         private bool ended;
29         private int maxBufferSize = DefaultMaxBufferSize;
30
31         /// <summary>Creates a stream backed by a default canonical parser.</summary>
32         public StreamParser() : this(new Parser()) { }
33
34         /// <summary>Creates a stream backed by <paramref name="parser"/>.</summary>
35         public StreamParser(Parser parser)
36         {
37             parser = parser ?? throw new ArgumentNullException(nameof(parser));
38             Reset();
39         }
40
41         /// <summary>Raised once for every completed link.</summary>
42         public event Action<Link<string>>? LinkParsed;
43
44         /// <summary>Whether emitted links are retained for <see cref="Finish()"/> and <see
45         ↪ cref="Drain"/>.</summary>
46         public bool Collect { get; set; } = true;
47
48         /// <summary>The largest unresolved record accepted by the stream.</summary>
49         public int MaxBufferSize
50         {
51             get => maxBufferSize;
52             set => maxBufferSize = value > 0
                    ? value

```

```

53         : throw new ArgumentOutOfRangeException(nameof(value), "Maximum buffer size must
↳ be positive.");
54     }
55
56     /// <summary>The point immediately after the last character written.</summary>
57     public StreamPosition Position => new(
58         offset,
59         line,
60         column,
61         buffer.Length + currentLine.Length);
62
63     /// <summary>Consumes a chunk and returns links made complete by it.</summary>
64     public IReadOnlyList<Link<string>> Write(string chunk)
65     {
66         ArgumentNullException.ThrowIfNull(chunk);
67         if (ended) throw new InvalidOperationException("Cannot write after Finish().");
68
69         var emitted = new List<Link<string>>();
70         foreach (var character in chunk)
71         {
72             currentLine.Append(character);
73             offset++;
74
75             if (character == '\n')
76             {
77                 buffer.Append(currentLine);
78                 currentLine.Clear();
79                 lineClassified = false;
80                 line++;
81                 column = 1;
82             }
83             else
84             {
85                 if (!lineClassified && character != ' ' && character != '\t' && character
↳ != '\r')
86                 {
87                     lineClassified = true;
88                     if (!(parser.Comments && character == Comments.Comment))
89                     {
90                         StartContentLine(LeadingSpaces(currentLine), emitted);
91                     }
92                 }
93                 column++;
94             }
95
96             if (buffer.Length + currentLine.Length > MaxBufferSize)
97             {
98                 throw new ArgumentException(
99                     $"Buffered record exceeds maximum size of {MaxBufferSize} characters.",
100                     nameof(chunk));
101             }
102         }
103         return emitted;
104     }
105
106     /// <summary>Finishes the stream and returns all undrained links.</summary>
107     public IReadOnlyList<Link<string>> Finish() => Finish(string.Empty);
108
109     /// <summary>Consumes a final chunk and finishes the stream.</summary>
110     public IReadOnlyList<Link<string>> Finish(string chunk)
111     {
112         if (ended)
113         {
114             if (chunk.Length > 0) throw new InvalidOperationException("Cannot write after
↳ Finish().");
115             return Collect ? links.ToArray() : Array.Empty<Link<string>>();
116         }
117
118         var emitted = chunk.Length == 0
119             ? new List<Link<string>>()
120             : new List<Link<string>>(Write(chunk));
121         var document = buffer.ToString() + currentLine;
122         if (document.Length > 0)
123         {
124             IList<Link<string>> parsed;
125             try
126             {
127                 parsed = parser.Parse(document);
128             }

```

```

129         catch (ParseException error)
130         {
131             throw new StreamParseException(
132                 error,
133                 segmentOffset + error.Offset,
134                 segmentLine + error.Line - 1,
135                 error.Column);
136         }
137         Publish(parsed, emitted);
138         AdvanceSegment(document);
139     }
140
141     buffer.Clear();
142     currentLine.Clear();
143     baseIndentation = null;
144     ended = true;
145     return Collect ? links.ToArray() : emitted;
146 }
147
148 /// <summary>Returns and forgets links retained since the previous call.</summary>
149 public IReadOnlyList<Link<string>> Drain()
150 {
151     var links = links.ToArray();
152     links.Clear();
153     return links;
154 }
155
156 /// <summary>Reuses the parser while preserving its options and event handlers.</summary>
157 public StreamParser Reset()
158 {
159     buffer.Clear();
160     currentLine.Clear();
161     baseIndentation = null;
162     lineClassified = false;
163     links.Clear();
164     offset = 0;
165     line = 1;
166     column = 1;
167     segmentOffset = 0;
168     segmentLine = 1;
169     ended = false;
170     return this;
171 }
172
173 /// <summary>Lazily parses a sequence of chunks as a native enumerable.</summary>
174 public static IEnumerable<Link<string>> ParseChunks(
175     IEnumerable<string> chunks,
176     Parser? parser = null)
177 {
178     var stream = new StreamParser(parser ?? new Parser()) { Collect = false };
179     foreach (var chunk in chunks)
180     {
181         foreach (var link in stream.Write(chunk)) yield return link;
182     }
183     foreach (var link in stream.Finish()) yield return link;
184 }
185
186 /// <summary>Lazily parses an asynchronous sequence of chunks.</summary>
187 public static async IAsyncEnumerable<Link<string>> ParseChunksAsync(
188     IAsyncEnumerable<string> chunks,
189     Parser? parser = null,
190     [EnumeratorCancellation] CancellationToken cancellationToken = default)
191 {
192     var stream = new StreamParser(parser ?? new Parser()) { Collect = false };
193     await foreach (var chunk in chunks.WithCancellation(cancellationToken))
194     {
195         foreach (var link in stream.Write(chunk)) yield return link;
196     }
197     foreach (var link in stream.Finish()) yield return link;
198 }
199
200 private void StartContentLine(int indentation, List<Link<string>> emitted)
201 {
202     if ( buffer.Length > 0 &&
203         baseIndentation.HasValue &&
204         indentation <= baseIndentation.Value &&
205         StructurallyComplete( buffer.ToString(), parser.Comments))
206     {
207         try
208         {

```

```

209         var document = buffer.ToString();
210         var parsed = parser.Parse(document);
211         Publish(parsed, emitted);
212         AdvanceSegment(document);
213         buffer.Clear();
214         baseIndentation = null;
215     }
216     catch (ParseException)
217     {
218         // A rejected prefix can become valid when more input arrives.
219     }
220 }
221
222     baseIndentation ??= indentation;
223 }
224
225 private void Publish(IEnumerable<Link<string>> links, List<Link<string>> emitted)
226 {
227     foreach (var link in links)
228     {
229         emitted.Add(link);
230         if (Collect) links.Add(link);
231         LinkParsed?.Invoke(link);
232     }
233 }
234
235 private void AdvanceSegment(string document)
236 {
237     segmentOffset += document.Length;
238     foreach (var character in document)
239     {
240         if (character == '\n') segmentLine++;
241     }
242 }
243
244 private static int LeadingSpaces(StringBuilder line)
245 {
246     var indentation = 0;
247     while (indentation < line.Length && line[indentation] == ' ') indentation++;
248     return indentation;
249 }
250
251 private static bool StructurallyComplete(string document, bool comments)
252 {
253     var depth = 0;
254     for (var position = 0; position < document.Length; position++)
255     {
256         var character = document[position];
257         if (Quotes.IsQuote(character) && Follows(document, position, BeforeReference))
258         {
259             var end = Quotes.ReferenceEnd(document, position);
260             if (end < 0) return false;
261             position = end - 1;
262             continue;
263         }
264         if (comments && character == Comments.Comment && Follows(document, position,
265 ↪ BeforeComment))
266         {
267             var newline = document.IndexOf('\n', position);
268             if (newline < 0) break;
269             position = newline;
270             continue;
271         }
272         if (character == '(') depth++;
273         else if (character == ')') depth--;
274     }
275     return depth == 0;
276 }
277
278 private static bool Follows(string document, int position, string allowed) =>
279     position == 0 || allowed.IndexOf(document[position - 1]) >= 0;
280 }
281
282 /// <summary>An absolute stream position and the unresolved record size.</summary>
283 public readonly record struct StreamPosition(int Offset, int Line, int Column, int Buffered);

```

## 1.15 ./Link.Foundation.Links.Notation/.cs

```
1 using System.Runtime.CompilerServices;
2
3 namespace Link.Foundation.Links.Notation
4 {
5     /// <summary>
6     /// A utility struct that provides convenient implicit conversions for creating links from
7     /// various tuple combinations.
8     /// The underscore name follows functional programming conventions for placeholder/utility
9     /// types.
10    /// </summary>
11    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
12    /// any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
13    public struct <TLinkAddress>
14    {
15        /// <summary>
16        /// Gets the underlying link represented by this utility struct.
17        /// </summary>
18        public readonly Link<TLinkAddress> Link;
19
20        /// <summary>
21        /// Initializes a new instance with the specified link.
22        /// </summary>
23        /// <param name="id">The link to wrap.</param>
24        [MethodImpl(MethodImplOptions.AggressiveInlining)]
25        public (Link<TLinkAddress> id) => Link = id;
26
27        /// <summary>
28        /// Implicitly converts a link to this utility struct.
29        /// </summary>
30        /// <param name="value">The link to convert.</param>
31        [MethodImpl(MethodImplOptions.AggressiveInlining)]
32        public static implicit operator <TLinkAddress>(Link<TLinkAddress> value) => new(value);
33
34        /// <summary>
35        /// Implicitly converts a link address to this utility struct.
36        /// </summary>
37        /// <param name="id">The link address to convert.</param>
38        [MethodImpl(MethodImplOptions.AggressiveInlining)]
39        public static implicit operator <TLinkAddress>(TLinkAddress id) => new(id);
40
41        /// <summary>
42        /// Implicitly converts a tuple of address and link to this utility struct.
43        /// </summary>
44        /// <param name="value">The tuple containing address and link.</param>
45        [MethodImpl(MethodImplOptions.AggressiveInlining)]
46        public static implicit operator <TLinkAddress>((TLinkAddress, Link<TLinkAddress>)
47        value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2 });
48
49        /// <summary>
50        /// Implicitly converts a tuple of address and two links to this utility struct.
51        /// </summary>
52        /// <param name="value">The tuple containing address and two links.</param>
53        [MethodImpl(MethodImplOptions.AggressiveInlining)]
54        public static implicit operator <TLinkAddress>((TLinkAddress, Link<TLinkAddress>,
55        Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2,
56        value.Item3 });
57
58        /// <summary>
59        /// Implicitly converts a tuple of address and three links to this utility struct.
60        /// </summary>
61        /// <param name="value">The tuple containing address and three links.</param>
62        [MethodImpl(MethodImplOptions.AggressiveInlining)]
63        public static implicit operator <TLinkAddress>((TLinkAddress, Link<TLinkAddress>,
64        Link<TLinkAddress>, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[]
65        { value.Item2, value.Item3, value.Item4 });
66
67        /// <summary>
68        /// Implicitly converts this utility struct back to a link.
69        /// </summary>
70        /// <param name="value">The utility struct to convert.</param>
71        [MethodImpl(MethodImplOptions.AggressiveInlining)]
72        public static implicit operator Link<TLinkAddress>( <TLinkAddress> value) => value.Link;
73    }
74 }
```

## 1.16 ./Link.Foundation.Links.Notation/id.cs

```
1 using System.Runtime.CompilerServices;
2
```

```

3 namespace Link.Foundation.Links.Notation
4 {
5     /// <summary>
6     /// A readonly struct that explicitly represents a link identifier/address.
7     /// This type is used to distinguish between regular values and explicit identifiers in link
8     /// </summary>
9     /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
10    /// any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
11    public readonly struct id<TLinkAddress>
12    {
13        /// <summary>
14        /// Gets the identifier value.
15        /// </summary>
16        public readonly TLinkAddress Id;
17
18        /// <summary>
19        /// Initializes a new identifier with the specified value.
20        /// </summary>
21        /// <param name="id">The identifier value.</param>
22        [MethodImpl(MethodImplOptions.AggressiveInlining)]
23        public id(TLinkAddress id) => Id = id;
24
25        /// <summary>
26        /// Explicitly converts a value to an identifier struct.
27        /// </summary>
28        /// <param name="id">The value to convert to an identifier.</param>
29        [MethodImpl(MethodImplOptions.AggressiveInlining)]
30        public static explicit operator id<TLinkAddress>(TLinkAddress id) => new(id);
31    }
32 }

```

#### 1.17 ./Link.Foundation.Links.Notation.Tests/ApiTests.cs

```

1 using System;
2 using System.Collections.Generic;
3 using Xunit;
4
5 namespace Link.Foundation.Links.Notation.Tests
6 {
7     public static class ApiTests
8     {
9         [Fact]
10        public static void IsRefTest()
11        {
12            // C# doesn't have separate Ref/Link types, but we can test simple link behavior
13            var simpleLink = new Link<string>("some value", null);
14            Assert.Equal("some value", simpleLink.Id);
15            Assert.Null(simpleLink.Values);
16        }
17
18        [Fact]
19        public static void IsLinkTest()
20        {
21            // Test link with values
22            var values = new List<Link<string>> { new Link<string>("child", null) };
23            var link = new Link<string>("id", values);
24            Assert.Equal("id", link.Id);
25            Assert.Single(link.Values!);
26            Assert.Equal("child", link.Values?[0].Id);
27        }
28
29        [Fact]
30        public static void IsRefEquivalentTest()
31        {
32            // C# doesn't have separate Ref/Link types, but we can test simple link behavior
33            var simpleLink = new Link<string>("some value", null);
34            Assert.Equal("some value", simpleLink.Id);
35            Assert.Null(simpleLink.Values);
36        }
37
38        [Fact]
39        public static void IsLinkEquivalentTest()
40        {
41            // Test link with values
42            var values = new List<Link<string>> { new Link<string>("child", null) };
43            var link = new Link<string>("id", values);
44            Assert.Equal("id", link.Id);
45            Assert.Single(link.Values!);
46            Assert.Equal("child", link.Values?[0].Id);
47        }
48    }
49 }

```

```

47     }
48
49     [Fact]
50     public static void EmptyLinkTest()
51     {
52         var link = new Link<string>(null, new List<Link<string>>());
53         var output = link.ToString();
54         Assert.Equal("()", output);
55     }
56
57     [Fact]
58     public static void SimpleLinkTest()
59     {
60         var input = "(1: 1 1)";
61         var parser = new Parser();
62         var parsed = parser.Parse(input);
63
64         // Validate regular formatting
65         var output = parsed.Format();
66         Assert.Contains("1:", output);
67         Assert.Contains("1", output);
68     }
69
70     [Fact]
71     public static void LinkWithSourceTargetTest()
72     {
73         var input = "(index: source target)";
74         var parser = new Parser();
75         var parsed = parser.Parse(input);
76
77         // Validate regular formatting
78         var output = parsed.Format();
79         Assert.Equal(input, output);
80     }
81
82     [Fact]
83     public static void LinkWithSourceTypeTargetTest()
84     {
85         var input = "(index: source type target)";
86         var parser = new Parser();
87         var parsed = parser.Parse(input);
88
89         // Validate regular formatting
90         var output = parsed.Format();
91         Assert.Equal(input, output);
92     }
93
94     [Fact]
95     public static void SingleLineFormatTest()
96     {
97         var input = "id: value1 value2";
98         var parser = new Parser();
99         var parsed = parser.Parse(input);
100
101         // The parser should handle single-line format
102         var output = parsed.Format();
103         Assert.Contains("id", output);
104         Assert.Contains("value1", output);
105         Assert.Contains("value2", output);
106     }
107
108     [Fact]
109     public static void QuotedReferencesTest()
110     {
111         var input = @"("quoted id": "value with spaces")";
112         var parser = new Parser();
113         var parsed = parser.Parse(input);
114
115         var output = parsed.Format();
116         Assert.Contains("quoted id", output);
117         Assert.Contains("value with spaces", output);
118     }
119
120     [Fact]
121     public static void QuotedReferencesParsingTest()
122     {
123         // Test that quoted references are parsed correctly
124         var input = @"("quoted id": "value with spaces")";
125         var parser = new Parser();

```

```

126     var parsed = parser.Parse(input);
127
128     // Verify parsing worked correctly
129     var output = parsed.Format();
130     Assert.Contains("quoted id", output);
131     Assert.Contains("value with spaces", output);
132 }
133
134 [Fact]
135 public static void IndentedIdSyntaxParsingTest()
136 {
137     // Test that indented ID syntax is parsed correctly
138     var indented = "id:\n value1\n value2";
139     var inline = "(id: value1 value2)";
140
141     var parser = new Parser();
142     var indentedParsed = parser.Parse(indented);
143     var inlineParsed = parser.Parse(inline);
144
145     // Both should produce equivalent structures
146     var indentedOutput = indentedParsed.Format();
147     var inlineOutput = inlineParsed.Format();
148     Assert.Equal(indentedOutput, inlineOutput);
149     Assert.Equal("(id: value1 value2)", indentedOutput);
150 }
151
152 [Fact]
153 public static void IndentedIdSyntaxRoundtripTest()
154 {
155     var input = "id:\n value1\n value2";
156     var parser = new Parser();
157     var parsed = parser.Parse(input);
158
159     // Validate that we can format with indented syntax using FormatOptions
160     var options = new FormatOptions
161     {
162         MaxInlineRefs = 1, // Force indentation with more than 1 ref
163         PreferInline = false
164     };
165     var output = parsed.Format(options);
166     Assert.Equal(input, output);
167 }
168
169 [Fact]
170 public static void MultipleIndentedIdSyntaxParsingTest()
171 {
172     // Test that multiple indented ID links are parsed correctly
173     var indented = "id1:\n a\n b\nid2:\n c\n d";
174     var inline = "(id1: a b)\n(id2: c d)";
175
176     var parser = new Parser();
177     var indentedParsed = parser.Parse(indented);
178     var inlineParsed = parser.Parse(inline);
179
180     // Both should produce equivalent structures
181     var indentedOutput = indentedParsed.Format();
182     var inlineOutput = inlineParsed.Format();
183     Assert.Equal(indentedOutput, inlineOutput);
184     Assert.Equal("(id1: a b)\n(id2: c d)", indentedOutput);
185 }
186
187 [Fact]
188 public static void MultipleIndentedIdSyntaxRoundtripTest()
189 {
190     var input = "id1:\n a\n b\nid2:\n c\n d";
191     var parser = new Parser();
192     var parsed = parser.Parse(input);
193
194     // Validate that we can format with indented syntax using FormatOptions
195     var options = new FormatOptions
196     {
197         MaxInlineRefs = 1, // Force indentation with more than 1 ref
198         PreferInline = false
199     };
200     var output = parsed.Format(options);
201     Assert.Equal(input, output);
202 }
203 }
204 }

```

## 1.18 ./Link.Foundation.Links.Notation.Tests/CommentsTests.cs

```

1  using System.Linq;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      /// <summary>
7      /// Conformance tests for line comments.
8      ///
9      /// https://github.com/link-foundation/links-notation/issues/301
10     ///
11     /// <c>#</c> starts a comment that runs to the end of the line, unless it sits
12     /// inside a token or inside a delimited reference. Parsers read comments by
13     /// default and can be told to treat <c>#</c> as an ordinary character again.
14     /// The table below is shared with the Rust, JavaScript, Python, Go, Java and
15     /// PHP suites, so a document written by one implementation reads the same in
16     /// all of them.
17     /// </summary>
18     public static class CommentsTests
19     {
20         /// <summary>
21         /// Renders a parsed node unambiguously: every reference is wrapped in
22         /// angle brackets.
23         /// </summary>
24         private static string Render(Link<string> node)
25         {
26             if (node.Values == null || node.Values.Count == 0)
27             {
28                 return "<" + (node.Id ?? "") + ">";
29             }
30             var head = node.Id == null ? "" : "<" + node.Id + ">: ";
31             return "(" + head + string.Join(" ", node.Values.Select(Render)) + ")";
32         }
33
34         private static string RenderedWith(Parser parser, string source) =>
35             string.Join("\n", parser.Parse(source).Select(Render));
36
37         private static string Rendered(string source) => RenderedWith(new Parser(), source);
38
39         private static void AssertParsesAs(string source, string expected)
40         {
41             Assert.Equal(expected, Rendered(source));
42         }
43
44         [Fact]
45         public static void ALineThatStartsWithAHashIsAComment()
46         {
47             AssertParsesAs("# a b\n", "");
48         }
49
50         [Fact]
51         public static void ACommentMayHoldAColon()
52         {
53             // The document from #301: prose with a colon used to be rejected.
54             AssertParsesAs("# a: b\n", "");
55         }
56
57         [Fact]
58         public static void ACommentMayHoldAnythingAtAll()
59         {
60             AssertParsesAs("# ) : ( \" ' ` #\n", "");
61         }
62
63         [Fact]
64         public static void ACommentEndsAtTheEndOfItsLine()
65         {
66             AssertParsesAs("# note\na: b\n", "<a>: <b>");
67         }
68
69         [Fact]
70         public static void ACommentMayFollowALink()
71         {
72             AssertParsesAs("a: b # why\n", "<a>: <b>");
73         }
74
75         [Fact]
76         public static void ACommentMayFollowAGroup()
77         {
78             AssertParsesAs("(a b) # why\n", "<a> <b>");
79         }
80     }
81 }

```

```

79     }
80
81     [Fact]
82     public static void ACommentNeedsNoClosingNewline()
83     {
84         AssertParsesAs("a: b # why", "<a>: <b>");
85     }
86
87     [Fact]
88     public static void ACommentLineInsideAnIndentedBlockIsSkipped()
89     {
90         Assert.Equal(
91             Rendered("parent\n  child\n"),
92             Rendered("parent\n  # what the child is for\n  child\n"));
93     }
94
95     [Fact]
96     public static void ACommentLineInsideAGroupIsSkipped()
97     {
98         Assert.Equal(
99             Rendered("\n  a\n  b\n\n"),
100             Rendered("\n  a\n  # why\n  b\n\n"));
101     }
102
103     [Fact]
104     public static void ALineOfSpacesSeparatesLinksTheWayAnEmptyLineDoes()
105     {
106         // Blanking a comment leaves a line of spaces behind, so such a line
107         // has to read the way an empty line does.
108         Assert.Equal(Rendered("a\n\nb\n"), Rendered("a\n  \nb\n"));
109     }
110
111     [Fact]
112     public static void ADocumentOfCommentsAloneHoldsNoLinks()
113     {
114         AssertParsesAs("# one\n# two\n", "");
115     }
116
117     [Fact]
118     public static void AHashInsideATokenIsAnOrdinaryCharacter()
119     {
120         AssertParsesAs("issue#1047\n", "<issue#1047>");
121     }
122
123     [Fact]
124     public static void AHashThatOpensATokenIsAnOrdinaryCharacter()
125     {
126         AssertParsesAs("a: b#c\n", "<a>: <b#c>");
127     }
128
129     [Fact]
130     public static void AHashInsideADelimitedReferenceIsContent()
131     {
132         AssertParsesAs("<\"# not a comment\" a\n", "<# not a comment> <a>");
133     }
134
135     [Fact]
136     public static void ACommentMayFollowADelimitedReference()
137     {
138         AssertParsesAs("<\"a\" # why\n", "<a>");
139     }
140
141     [Fact]
142     public static void AHashInsideAMultiLineDelimitedReferenceIsContent()
143     {
144         AssertParsesAs("<\"a # b\nc\" d\n", "<a # b\nc> <d>");
145     }
146
147     [Fact]
148     public static void CommentsAreOnByDefault()
149     {
150         Assert.True(new Parser().Comments);
151         AssertParsesAs("# a b\n", "");
152     }
153
154     [Fact]
155     public static void AParserWithoutCommentsKeepsTheHash()
156     {
157         Assert.Equal("<#> <a> <b>", RenderedWith(new Parser(comments: false), "# a b\n"));

```

```

158     }
159
160     [Fact]
161     public static void BlankingACommentKeepsTheLengthOfTheDocument()
162     {
163         Assert.Equal("a: b      \n", Comments.StripComments("a: b # why\n"));
164         Assert.Equal("\n# kept\n", Comments.StripComments("\n# kept\n"));
165         Assert.Equal("issue#1047\n", Comments.StripComments("issue#1047\n"));
166         Assert.Equal("      \n", Comments.StripComments("# why\n"));
167     }
168
169     [Fact]
170     public static void ACommentDoesNotMoveThePositionALaterErrorIsReportedAt()
171     {
172         var error = Assert.Throws<ParseException>(
173             () => new Parser().Parse("# a comment\nstage: rust: nextest\n"));
174
175         Assert.Equal(2, error.Line);
176         Assert.Equal(12, error.Column);
177     }
178
179     [Fact]
180     public static void AReferenceThatBeginsWithAHashIsWrittenQuoted()
181     {
182         // Without the quotes the document would read as `a` followed by a comment.
183         var document = new Link<string>(new Link<string>("a"), new
↪ Link<string>("#tag")).ToString();
184
185         Assert.Equal("(a '#tag')", document);
186         Assert.Equal("<a> <#tag>", Rendered(document));
187     }
188
189     [Fact]
190     public static void AHashThatCannotOpenACommentIsLeftUnquoted()
191     {
192         Assert.Equal("issue#1047", Link<string>.EscapeReference("issue#1047"));
193         Assert.Equal("#'", Link<string>.EscapeReference("#"));
194         Assert.Equal("#ff0000'", Link<string>.EscapeReference("#ff0000"));
195     }
196 }
197 }

```

## 1.19 ./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class EdgeCaseParserTests
7      {
8          [Fact]
9          public static void EmptyLinkTest()
10         {
11             var source = @": ";
12             var parser = new Parser();
13             // Standalone ':' is now forbidden and should throw an exception
14             Assert.Throws<ParseException>(() => parser.Parse(source));
15         }
16
17         [Fact]
18         public static void EmptyLinkWithParenthesesTest()
19         {
20             var source = @"() ";
21             var target = @"() ";
22             var parser = new Parser();
23             var links = parser.Parse(source);
24             var formattedLinks = links.Format();
25             Assert.Equal(target, formattedLinks);
26         }
27
28         [Fact]
29         public static void EmptyLinkWithEmptySelfReferenceTest()
30         {
31             var source = @"(:) ";
32             var parser = new Parser();
33             // '(:)' is now forbidden and should throw an exception
34             Assert.Throws<ParseException>(() => parser.Parse(source));
35         }
36     }

```

```

37 [Fact]
38 public static void AllFeaturesTest()
39 {
40     // Test single-line link with id
41     var input = "id: value1 value2";
42     var result = new Parser().Parse(input);
43     Assert.NotEmpty(result);
44
45     // Test multi-line link with id
46     input = "(id: value1 value2)";
47     result = new Parser().Parse(input);
48     Assert.NotEmpty(result);
49
50     // Test link without id (single-line) - now forbidden
51     input = ": value1 value2";
52     Assert.Throws<ParseException>(() => new Parser().Parse(input));
53
54     // Test link without id (multi-line) - now forbidden
55     input = "(: value1 value2)";
56     Assert.Throws<ParseException>(() => new Parser().Parse(input));
57
58     // Test singlet link
59     input = "(singlet)";
60     result = new Parser().Parse(input);
61     Assert.Single(result);
62     Assert.Null(result[0].Id);
63     Assert.NotNull(result[0].Values);
64     Assert.Single(result[0].Values!);
65     Assert.Equal("singlet", result[0].Values?[0].Id);
66     Assert.Null(result[0].Values?[0].Values);
67
68     // Test value link
69     input = "(value1 value2 value3)";
70     result = new Parser().Parse(input);
71     Assert.NotEmpty(result);
72
73     // Test quoted references
74     input = @"("id with spaces": "value with spaces")";
75     result = new Parser().Parse(input);
76     Assert.NotEmpty(result);
77
78     // Test single-quoted references
79     input = "('id': 'value')";
80     result = new Parser().Parse(input);
81     Assert.NotEmpty(result);
82
83     // Test nested links
84     input = "(outer: (inner: value))";
85     result = new Parser().Parse(input);
86     Assert.NotEmpty(result);
87 }

```

```

88 [Fact]
89 public static void TestSingletLinks()
90 {
91     // Test singlet (1)
92     var input = "(1)";
93     var result = new Parser().Parse(input);
94     Assert.Single(result);
95     Assert.Null(result[0].Id);
96     Assert.NotNull(result[0].Values);
97     Assert.Single(result[0].Values!);
98     Assert.Equal("1", result[0].Values?[0].Id);
99     Assert.Null(result[0].Values?[0].Values);
100
101     // Test (1 2)
102     input = "(1 2)";
103     result = new Parser().Parse(input);
104     Assert.Single(result);
105     Assert.Null(result[0].Id);
106     Assert.NotNull(result[0].Values);
107     Assert.Equal(2, result[0].Values?.Count);
108     Assert.Equal("1", result[0].Values?[0].Id);
109     Assert.Null(result[0].Values?[0].Values);
110     Assert.Equal("2", result[0].Values?[1].Id);
111     Assert.Null(result[0].Values?[1].Values);
112
113     // Test (1 2 3)
114     input = "(1 2 3)";
115

```

```

116     result = new Parser().Parse(input);
117     Assert.Single(result);
118     Assert.Null(result[0].Id);
119     Assert.NotNull(result[0].Values);
120     Assert.Equal(3, result[0].Values?.Count);
121     Assert.Equal("1", result[0].Values?[0].Id);
122     Assert.Null(result[0].Values?[0].Values);
123     Assert.Equal("2", result[0].Values?[1].Id);
124     Assert.Null(result[0].Values?[1].Values);
125     Assert.Equal("3", result[0].Values?[2].Id);
126     Assert.Null(result[0].Values?[2].Values);
127
128     // Test (1 2 3 4)
129     input = "(1 2 3 4)";
130     result = new Parser().Parse(input);
131     Assert.Single(result);
132     Assert.Null(result[0].Id);
133     Assert.NotNull(result[0].Values);
134     Assert.Equal(4, result[0].Values?.Count);
135     Assert.Equal("1", result[0].Values?[0].Id);
136     Assert.Null(result[0].Values?[0].Values);
137     Assert.Equal("2", result[0].Values?[1].Id);
138     Assert.Null(result[0].Values?[1].Values);
139     Assert.Equal("3", result[0].Values?[2].Id);
140     Assert.Null(result[0].Values?[2].Values);
141     Assert.Equal("4", result[0].Values?[3].Id);
142     Assert.Null(result[0].Values?[3].Values);
143 }
144
145 [Fact]
146 public static void EmptyDocumentTest()
147 {
148     var input = "";
149     // Empty document should return empty list
150     var result = new Parser().Parse(input);
151     Assert.Empty(result);
152 }
153
154 [Fact]
155 public static void WhitespaceOnlyTest()
156 {
157     var input = " \n \n ";
158     // Whitespace-only document should return empty list (similar to empty document)
159     var result = new Parser().Parse(input);
160     Assert.Empty(result);
161 }
162
163 [Fact]
164 public static void EmptyLinksTest()
165 {
166     var input = "()";
167     var result = new Parser().Parse(input);
168     Assert.NotEmpty(result);
169
170     // '(:)' is now forbidden
171     input = "(:)";
172     Assert.Throws<ParseException>(() => new Parser().Parse(input));
173
174     input = "(id:)";
175     result = new Parser().Parse(input);
176     Assert.NotEmpty(result);
177 }
178
179 [Fact]
180 public static void InvalidInputTest()
181 {
182     var input = "(invalid";
183     // Unclosed parentheses should throw an exception
184     Assert.Throws<ParseException>(() => new Parser().Parse(input));
185 }
186 }
187

```

## 1.20 ./Link.Foundation.Links.Notation.Tests/EmptyReferenceTests.cs

```

1 using System.Collections.Generic;
2 using System.Linq;
3 using Xunit;
4
5 namespace Link.Foundation.Links.Notation.Tests

```

```

6 {
7     /// <summary>
8     /// Conformance tests for the empty reference.
9     ///
10    /// https://github.com/link-foundation/links-notation/issues/288
11    ///
12    /// A bare delimiter pair is the empty reference. The three delimiters
13    /// <c>"</c>, <c>'</c> and <c>`</c> behave identically, and every longer
14    /// n-quote run keeps the meaning it already had. The table below is shared
15    /// with the Rust, JavaScript, Python, Go, Java and PHP suites, so a document
16    /// written by one implementation reads the same in all of them.
17    /// </summary>
18    public static class EmptyReferenceTests
19    {
20        /// <summary>
21        /// Renders a parsed node unambiguously: every reference is wrapped in
22        /// angle brackets so an empty one is visible as < >.
23        /// </summary>
24        private static string Render(Link<string> node)
25        {
26            if (node.Values == null || node.Values.Count == 0)
27            {
28                return "<" + (node.Id ?? "") + ">";
29            }
30            var head = node.Id == null ? "" : "<" + node.Id + ">: ";
31            return "(" + head + string.Join(" ", node.Values.Select(Render)) + ")";
32        }
33
34        private static string Rendered(string source)
35        {
36            var links = new Parser().Parse(source);
37            return string.Join("\n", links.Select(Render));
38        }
39
40        private static void AssertParsesAs(string source, string expected)
41        {
42            Assert.Equal(expected, Rendered(source));
43        }
44
45        [Fact]
46        public static void BareDelimiterPairIsTheEmptyReference()
47        {
48            AssertParsesAs("(a \" b)", "<a> <> <b>");
49        }
50
51        [Fact]
52        public static void EveryDelimiterStyleYieldsTheSameEmptyReference()
53        {
54            AssertParsesAs("(a \" b)", "<a> <> <b>");
55            AssertParsesAs("(a ' b)", "<a> <> <b>");
56            AssertParsesAs("(a ` b)", "<a> <> <b>");
57        }
58
59        [Fact]
60        public static void AdjacentEmptyReferencesStaySeparate()
61        {
62            AssertParsesAs("(a \" \" b)", "<a> <> <> <b>");
63            AssertParsesAs("(a ' ' b)", "<a> <> <> <b>");
64            AssertParsesAs("(a ` ` b)", "<a> <> <> <b>");
65            AssertParsesAs("(a \" \" b)", "<a> <> <> <b>");
66        }
67
68        [Fact]
69        public static void NestedEmptyReferencesParse()
70        {
71            AssertParsesAs("(\" \" (\" \" 1))", "<(> (> <1>))");
72            AssertParsesAs("(\" \" (' 1))", "<(> (> <1>))");
73            AssertParsesAs("(\" \" (\"x\" 1))", "<(<x> (> <1>))");
74            AssertParsesAs("(\" \" (\"x\" 1))", "<(> (<x> <1>))");
75            AssertParsesAs("(\" \" x (\" \" 1))", "<(> <x> (> <1>))");
76            AssertParsesAs("(\" \" 1 (\" \" 1))", "<(> <1> (> <1>))");
77        }
78
79        [Fact]
80        public static void EmptyReferenceIsValidAsAnId()
81        {
82            AssertParsesAs("(\" \": 1)", "<(>: <1>)");
83            AssertParsesAs("(o: (\" \" (o: (\" \" 1))))", "<(<o>: (> (<o>: (> <1>))))");

```



```

32     }
33
34     [Fact]
35     public void FormatWithMaxInlineRefsTest()
36     {
37         // Create a config with MaxInlineRefs=3 (should trigger indentation with 4 refs)
38         var config = new FormatConfig
39         {
40             MaxInlineRefs = 3,
41             PreferInline = false
42         };
43
44         // Verify config is set correctly
45         Assert.Equal(3, config.MaxInlineRefs);
46         Assert.True(config.ShouldIndentByRefCount(4));
47         // Note: Full formatting integration would test actual output here
48     }
49
50     [Fact]
51     public void FormatWithConsecutiveGroupingTest()
52     {
53         // Create a config with consecutive grouping enabled
54         var config = new FormatConfig
55         {
56             GroupConsecutive = true
57         };
58
59         // Verify config is set correctly
60         Assert.True(config.GroupConsecutive);
61         // Note: Full formatting integration would test grouping behavior here
62     }
63
64     [Fact]
65     public void FormatConfigCustomIndentTest()
66     {
67         var config = new FormatConfig
68         {
69             MaxInlineRefs = 3,
70             PreferInline = false,
71             IndentString = "    " // 4 spaces
72         };
73
74         Assert.Equal("    ", config.IndentString);
75     }
76
77     [Fact]
78     public void FormatConfigLessParenthesesTest()
79     {
80         var config = new FormatConfig
81         {
82             LessParentheses = true
83         };
84
85         Assert.True(config.LessParentheses);
86     }
87
88     [Fact]
89     public void RoundtripWithLineLengthFormattingTest()
90     {
91         // This test would require full format integration
92         // For now, just verify config creation
93         var config = new FormatConfig
94         {
95             MaxInlineRefs = 2,
96             PreferInline = false
97         };
98
99         Assert.Equal(2, config.MaxInlineRefs);
100     }
101
102     [Fact]
103     public void ShouldIndentByLengthTest()
104     {
105         var config = new FormatConfig
106         {
107             IndentLongLines = true,
108             MaxLineLength = 80
109         };
110
111         Assert.False(config.ShouldIndentByLength("short"));

```

```

112         Assert.True(config.ShouldIndentByLength(new string('a', 100)));
113     }
114     - - -
115     [Fact] -
116     public void ShouldIndentByRefCountTest()
117     {
118         -
119         var config = new FormatConfig
120         {
121             MaxInlineRefs = 3
122         };
123
124         Assert.False(config.ShouldIndentByRefCount(2));
125         Assert.False(config.ShouldIndentByRefCount(3));
126         Assert.True(config.ShouldIndentByRefCount(4));
127     }
128 }

```

## 1.22 ./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs

```

1  using Xunit;
2  using System.Collections.Generic;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public class IndentationConsistencyTests
7      {
8          [Fact]
9          public void LeadingSpacesVsNoLeadingSpaces()
10         {
11             // Example with 2 leading spaces (from issue #135)
12             var withLeading = @" TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
13 TELEGRAM ALLOWED CHATS:
14     -1002975819706
15     -1002861722681
16 TELEGRAM HIVE OVERRIDES:
17     --all-issues
18     --once
19 TELEGRAM BOT VERBOSE: true";
20
21             // Example without leading spaces (from issue #135)
22             var withoutLeading = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
23 TELEGRAM ALLOWED CHATS:
24     -1002975819706
25     -1002861722681
26 TELEGRAM HIVE OVERRIDES:
27     --all-issues
28     --once
29 TELEGRAM BOT VERBOSE: true";
30
31             var resultWith = new Parser().Parse(withLeading);
32             var resultWithout = new Parser().Parse(withoutLeading);
33
34             // Compare the entire formatted output (complete round trip test)
35             Assert.Equal(resultWithout.Format(), resultWith.Format());
36         }
37
38         [Fact]
39         public void TwoSpacesVsFourSpacesIndentation()
40         {
41             // Example with 2 spaces per level
42             var twoSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
43 TELEGRAM ALLOWED CHATS:
44     -1002975819706
45     -1002861722681
46 TELEGRAM HIVE OVERRIDES:
47     --all-issues
48     --once
49     --auto-fork
50     --skip-issues-with-prs
51     --attach-logs
52     --verbose
53     --no-tool-check
54 TELEGRAM SOLVE OVERRIDES:
55     --auto-fork
56     --auto-continue
57     --attach-logs
58     --verbose
59     --no-tool-check
60 TELEGRAM BOT VERBOSE: true";
61
62             // Example with 4 spaces per level
63             var fourSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'

```

```

64 TELEGRAM ALLOWED CHATS:
65     -1002975819706
66     -1002861722681
67 TELEGRAM HIVE OVERRIDES:
68     --all-issues
69     --once
70     --auto-fork
71     --skip-issues-with-prs
72     --attach-logs
73     --verbose
74     --no-tool-check
75 TELEGRAM SOLVE OVERRIDES:
76     --auto-fork
77     --auto-continue
78     --attach-logs
79     --verbose
80     --no-tool-check
81 TELEGRAM BOT VERBOSE: true";
82
83     var resultTwo = new Parser().Parse(twoSpaces);
84     var resultFour = new Parser().Parse(fourSpaces);
85
86     // Compare the entire formatted output (complete round trip test)
87     Assert.Equal(resultFour.Format(), resultTwo.Format());
88 }
89
90 [Fact]
91 public void SimpleTwoVsFourSpacesIndentation()
92 {
93     // Simple example with 2 spaces
94     var twoSpaces = @"parent:
95 child1
96 child2";
97
98     // Simple example with 4 spaces
99     var fourSpaces = @"parent:
100 child1
101 child2";
102
103     var resultTwo = new Parser().Parse(twoSpaces);
104     var resultFour = new Parser().Parse(fourSpaces);
105
106     // Compare the entire formatted output (complete round trip test)
107     Assert.Equal(resultFour.Format(), resultTwo.Format());
108 }
109
110 [Fact]
111 public void ThreeLevelNestingWithDifferentIndentation()
112 {
113     // Three levels with 2 spaces
114     var twoSpaces = @"level1:
115     level2:
116         level3a
117         level3b
118     level2b";
119
120     // Three levels with 4 spaces
121     var fourSpaces = @"level1:
122     level2:
123         level3a
124         level3b
125     level2b";
126
127     var resultTwo = new Parser().Parse(twoSpaces);
128     var resultFour = new Parser().Parse(fourSpaces);
129
130     // Compare the entire formatted output (complete round trip test)
131     Assert.Equal(resultFour.Format(), resultTwo.Format());
132 }
133 }
134 }

```

### 1.23 ./Link.Foundation.Links.Notation.Tests/IndentedIdNestedValuesTests.cs

```

1 using Xunit;
2
3 namespace Link.Foundation.Links.Notation.Tests
4 {
5     public class IndentedIdNestedValuesTests
6     {
7         [Fact]
8         public void IndentedIdRetainsNestedValues()
9         {

```

```

10         var cases = new[]
11         {
12             new[] { "outer:\n inner:\n value1\n value2\n value3", "(outer: (inner:
↪ value1 value2) value3)" },
13             new[] { "statement:\n subject:\n I\n verb:\n love\n object:\n you:\n
↪ very\n much", "(statement: (subject: I) (verb: love) (object: (you: very much)))"
↪ },
14             new[] { "document:\n (metadata: title author date)\n content:\n
↪ paragraph1\n (paragraph2: text (with: nested structure))\n paragraph3", "(document:
↪ (metadata: title author date) (content: paragraph1 (paragraph2: text (with: nested
↪ structure)) paragraph3))" },
15             new[] { "image:\n file: .gitpod.Dockerfile\n context: ./docker-content",
↪ "(image: (file: .gitpod.Dockerfile) (context: ./docker-content))" },
16             new[] { "level1:\n level2:\n level3a\n level3b\n level2b", "(level1:
↪ (level2: level3a level3b) level2b)" },
17             new[] { "root:\n child1\n child2\n grandchild", "(root: child1 (child2
↪ grandchild))" },
18         };
19
20         var parser = new Parser();
21         foreach (var testCase in cases)
22         {
23             Assert.Equal(testCase[1], parser.Parse(testCase[0]).Format());
24         }
25     }
26 }
27 }

```

## 1.24 ./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs

```

1 using System;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     public static class IndentedIdSyntaxTests
7     {
8         [Fact]
9         public static void BasicIndentedIdSyntaxTest()
10         {
11             var indentedSyntax = @"3:
12     papa
13     loves
14     mama";
15
16             var inlineSyntax = "(3: papa loves mama)";
17
18             var parser = new Parser();
19             var indentedResult = parser.Parse(indentedSyntax);
20             var inlineResult = parser.Parse(inlineSyntax);
21
22             var indentedFormatted = indentedResult.Format();
23             var inlineFormatted = inlineResult.Format();
24
25             Assert.Equal(inlineFormatted, indentedFormatted);
26             Assert.Equal("(3: papa loves mama)", indentedFormatted);
27
28             [Fact]
29             public static void IndentedIdSyntaxWithSingleValueTest()
30             {
31                 var input = @"greeting:
32     hello";
33
34                 var parser = new Parser();
35                 var result = parser.Parse(input);
36                 var formatted = result.Format();
37
38                 Assert.Equal("(greeting: hello)", formatted);
39                 Assert.Single(result);
40                 Assert.Equal("greeting", result[0].Id);
41                 Assert.Single(result[0].Values!);
42                 Assert.Equal("hello", result[0].Values![0].Id);
43             }
44
45             [Fact]
46             public static void IndentedIdSyntaxWithMultipleValuesTest()
47             {
48                 var input = @"action:
49     run
50     fast

```

```

51     now";
52
53     var parser = new Parser();
54     var result = parser.Parse(input);
55     var formatted = result.Format();
56
57     Assert.Equal("(action: run fast now)", formatted);
58     Assert.Single(result);
59     Assert.Equal("action", result[0].Id);
60     Assert.Equal(3, result[0].Values!.Count);
61 }
62
63 [Fact]
64 public static void IndentedIdSyntaxWithNumericIdTest()
65 {
66     var input = @"42:
67 answer
68 to
69 everything";
70
71     var parser = new Parser();
72     var result = parser.Parse(input);
73     var formatted = result.Format();
74
75     Assert.Equal("(42: answer to everything)", formatted);
76 }
77
78 [Fact]
79 public static void IndentedIdSyntaxWithQuotedIdTest()
80 {
81     var input = @"""complex id"":
82 value1
83 value2";
84
85     var parser = new Parser();
86     var result = parser.Parse(input);
87     var formatted = result.Format();
88
89     Assert.Equal("( 'complex id': value1 value2)", formatted);
90 }
91
92 [Fact]
93 public static void MultipleIndentedIdLinksTest()
94 {
95     var input = @"first:
96 a
97 b
98 second:
99 c
100 d";
101
102     var parser = new Parser();
103     var result = parser.Parse(input);
104     var formatted = result.Format();
105
106     Assert.Equal(2, result.Count);
107     Assert.Contains("(first: a b)", formatted);
108     Assert.Contains("(second: c d)", formatted);
109 }
110
111 [Fact]
112 public static void MixedIndentedAndRegularSyntaxTest()
113 {
114     var input = @"first:
115 a
116 b
117 (second: c d)
118 third value";
119
120     var parser = new Parser();
121     var result = parser.Parse(input);
122
123     Assert.Equal(3, result.Count);
124
125     var formatted = result.Format();
126     Assert.Contains("(first: a b)", formatted);
127     Assert.Contains("(second: c d)", formatted);
128     Assert.Contains("third value", formatted);
129 }
130

```

```

131     [Fact]
132     public static void UnsupportedColonOnlySyntaxShouldFailTest()
133     {
134         var input = @"
135     papa
136     loves
137     mama";
138
139         var parser = new Parser();
140         Assert.Throws<ParseException>(() => parser.Parse(input));
141     }
142
143     [Fact]
144     public static void EmptyIndentedIdShouldWorkTest()
145     {
146         var input = "empty:";
147
148         var parser = new Parser();
149         var result = parser.Parse(input);
150
151         Assert.Single(result);
152         Assert.Equal("empty", result[0].Id);
153         var emptyValues = result[0].Values;
154         Assert.True(emptyValues == null || emptyValues.Count == 0);
155
156         var formatted = result.Format();
157         Assert.Equal("(empty)", formatted);
158     }
159
160     [Fact]
161     public static void EquivalenceTestComprehensiveTest()
162     {
163         var testCases = new[]
164         {
165             new { Indented = "test:\n one", Inline = "(test: one)" },
166             new { Indented = "x:\n a\n b\n c", Inline = "(x: a b c)" },
167             new { Indented = "\"quoted\":\n value", Inline = "\"(quoted\": value)" }
168         };
169
170         var parser = new Parser();
171
172         foreach (var testCase in testCases)
173         {
174             var indentedResult = parser.Parse(testCase.Indented);
175             var inlineResult = parser.Parse(testCase.Inline);
176
177             var indentedFormatted = indentedResult.Format();
178             var inlineFormatted = inlineResult.Format();
179
180             Assert.Equal(inlineFormatted, indentedFormatted);
181         }
182     }
183
184     [Fact]
185     public static void IndentedIdWithDeeperNestingTest()
186     {
187         var input = @"
188     child1
189     child2
190     grandchild";
191
192         var parser = new Parser();
193         var result = parser.Parse(input);
194
195         Assert.NotEmpty(result);
196
197         var rootLink = result[0];
198         Assert.Equal("root", rootLink.Id);
199         Assert.Equal(2, rootLink.Values!.Count);
200         Assert.Equal("(root: child1 (child2 grandchild))", result.Format());
201     }
202 }
203

```

## 1.25 ./Link.Foundation.Links.Notation.Tests/LinkTests.cs

```

1  using System;
2  using Xunit;
3  using System.Collections.Generic;
4
5  namespace Link.Foundation.Links.Notation.Tests

```

```

6 {
7     public static class LinkTests
8     {
9         [Fact]
10        public static void LinkConstructorWithIdOnlyTest()
11        {
12            var link = new Link<string>("test", null);
13            Assert.Equal("test", link.Id);
14            Assert.Null(link.Values);
15        }
16
17        [Fact]
18        public static void LinkConstructorWithIdAndValuesTest()
19        {
20            var values = new List<Link<string>> { new Link<string>("value1"), new
↪ Link<string>("value2") };
21            var link = new Link<string>("parent", values);
22            Assert.Equal("parent", link.Id);
23            Assert.Equal(2, link.Values?.Count);
24        }
25
26        [Fact]
27        public static void LinkToStringWithIdOnlyTest()
28        {
29            var link = new Link<string>("test");
30            Assert.Equal("(test)", link.ToString());
31        }
32
33        [Fact]
34        public static void LinkToStringWithValuesOnlyTest()
35        {
36            var values = new List<Link<string>> { new Link<string>("value1"), new
↪ Link<string>("value2") };
37            var link = new Link<string>(null, values);
38            Assert.Equal("(value1 value2)", link.ToString());
39        }
40
41        [Fact]
42        public static void LinkToStringWithIdAndValuesTest()
43        {
44            var values = new List<Link<string>> { new Link<string>("child1"), new
↪ Link<string>("child2") };
45            var link = new Link<string>("parent", values);
46            Assert.Equal("(parent: child1 child2)", link.ToString());
47        }
48
49        [Fact]
50        public static void LinkEqualsTest()
51        {
52            var link1 = new Link<string>("test");
53            var link2 = new Link<string>("test");
54            var link3 = new Link<string>("other");
55
56            Assert.Equal(link1, link2);
57            Assert.NotEqual(link1, link3);
58        }
59
60        [Fact]
61        public static void LinkCombineTest()
62        {
63            var link1 = new Link<string>("first");
64            var link2 = new Link<string>("second");
65            var combined = link1.Combine(link2);
66
67            Assert.Null(combined.Id);
68            Assert.Equal(2, combined.Values?.Count);
69            Assert.Equal("first", combined.Values?[0].Id);
70            Assert.Equal("second", combined.Values?[1].Id);
71        }
72
73        [Fact]
74        public static void LinkEscapeReferenceSimpleTest()
75        {
76            Assert.Equal("test", Link<string>.EscapeReference("test"));
77            Assert.Equal("test123", Link<string>.EscapeReference("test123"));
78        }
79
80        [Fact]

```

```

81 public static void LinkEscapeReferenceWithSpecialCharactersTest()
82 {
83     Assert.Equal("'has space'", Link<string>.EscapeReference("has space"));
84     Assert.Equal("'has:colon'", Link<string>.EscapeReference("has:colon"));
85     Assert.Equal("'has(paren)'", Link<string>.EscapeReference("has(paren)"));
86     Assert.Equal("'has)paren'", Link<string>.EscapeReference("has)paren"));
87     Assert.Equal("\\'has'quote\\", Link<string>.EscapeReference("has'quote"));
88     Assert.Equal("'has\\'quote'", Link<string>.EscapeReference("has\\'quote"));
89 }
90
91 [Fact]
92 public static void LinkSimplifyTest()
93 {
94     // Test simplify with empty values
95     var link1 = new Link<string>("test", null);
96     var simplified1 = link1.Simplify();
97     Assert.Equal(link1, simplified1);
98
99     // Test simplify with single value
100    var link2 = new Link<string>(null, new List<Link<string>> { new
↪ Link<string>("single", null) });
101    var simplified2 = link2.Simplify();
102    Assert.Equal("single", simplified2.Id);
103    Assert.Null(simplified2.Values);
104 }
105 }
106 }

```

## 1.26 ./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs

```

1 using System;
2 using System.Collections.Generic;
3 using Xunit;
4
5 namespace Link.Foundation.Links.Notation.Tests
6 {
7     public static class LinksGroupTests
8     {
9         [Fact]
10        public static void LinksGroupConstructorTest()
11        {
12            var element = new Link<string>("root", null);
13            var groups = new List<LinksGroup<string>>
14            {
15                new LinksGroup<string>(new Link<string>("child1", null)),
16                new LinksGroup<string>(new Link<string>("child2", null))
17            };
18            var group = new LinksGroup<string>(element, groups);
19
20            Assert.Equal(element, group.Link);
21            Assert.Equal(groups, group.Groups);
22        }
23
24        [Fact]
25        public static void LinksGroupToListFlattensStructureTest()
26        {
27            var root = new Link<string>("root", null);
28            var child1 = new Link<string>("child1", null);
29            var child2 = new Link<string>("child2", null);
30            var grandchild = new Link<string>("grandchild", null);
31
32            var childGroup = new LinksGroup<string>(child2, new List<LinksGroup<string>>
33            {
34                new LinksGroup<string>(grandchild)
35            });
36
37            var group = new LinksGroup<string>(root, new List<LinksGroup<string>>
38            {
39                new LinksGroup<string>(child1),
40                childGroup
41            });
42
43            var list = group.ToLinksList();
44            // The C# implementation creates a hierarchical structure
45            // root, root.Combine(child1), root.Combine(child2),
↪ root.Combine(child2).Combine(grandchild)
46            Assert.Equal(4, list.Count);
47            Assert.Equal(root, list[0]);
48        }
49    }

```

```

50 [Fact]
51 public static void LinksGroupAppendToLinksListTest()
52 {
53     var element = new Link<string>("root", null);
54     var children = new List<LinksGroup<string>>
55     {
56         new LinksGroup<string>(new Link<string>("child1", null)),
57         new LinksGroup<string>(new Link<string>("child2", null))
58     };
59     var group = new LinksGroup<string>(element, children);
60
61     var list = new List<Link<string>>();
62     group.AppendToLinksList(list);
63
64     Assert.Equal(3, list.Count);
65     Assert.Equal(element, list[0]);
66 }
67
68 [Fact]
69 public static void LinksGroupToStringTest()
70 {
71     var element = new Link<string>("root", null);
72     var children = new List<LinksGroup<string>>
73     {
74         new LinksGroup<string>(new Link<string>("child1", null)),
75         new LinksGroup<string>(new Link<string>("child2", null))
76     };
77     var group = new LinksGroup<string>(element, children);
78
79     var str = group.ToString();
80     Assert.Contains("root", str);
81     Assert.Contains("child1", str);
82     Assert.Contains("child2", str);
83 }
84
85 [Fact]
86 public static void LinksGroupConstructorEquivalentTest()
87 {
88     // Test creating a LinksGroup structure in an equivalent way
89     var root = new Link<string>("root", null);
90     var children = new List<Link<string>>
91     {
92         new Link<string>("child1", null),
93         new Link<string>("child2", null)
94     };
95
96     // Create a group with an id
97     var groupElement = new Link<string>("group", null);
98     var childGroups = new List<LinksGroup<string>>
99     {
100         new LinksGroup<string>(root),
101         new LinksGroup<string>(children[0]),
102         new LinksGroup<string>(children[1])
103     };
104     var group = new LinksGroup<string>(groupElement, childGroups);
105
106     Assert.Equal("group", group.Link.Id);
107     Assert.Equal(3, group.Groups!.Count);
108     Assert.Equal(root, group.Groups[0].Link);
109 }
110 }
111 }

```

## 1.27 ./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs

```

1 using System;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     public static class MixedIndentationModesTests
7     {
8         [Fact]
9         public static void HeroExampleMixedModesTest()
10         {
11             var input = @"empInfo
12 employees:
13 (
14     name (James Kirk)
15     age 40

```

```

16     )
17     (
18         name (Jean-Luc Picard)
19         age 45
20     )
21     (
22         name (Wesley Crusher)
23         age 27
24     )";
25
26     var parser = new Parser();
27     var result = parser.Parse(input);
28
29     Assert.NotEmpty(result);
30     var formatted = result.Format();
31     Assert.Contains("empInfo", formatted);
32     Assert.Contains("employees", formatted);
33     Assert.Contains("James Kirk", formatted);
34     Assert.Contains("Jean-Luc Picard", formatted);
35     Assert.Contains("Wesley Crusher", formatted);
36 }
37
38 [Fact]
39 public static void HeroExampleAlternativeFormatTest()
40 {
41     var input = @"empInfo
42 (
43     employees:
44     (
45         name (James Kirk)
46         age 40
47     )
48     (
49         name (Jean-Luc Picard)
50         age 45
51     )
52     (
53         name (Wesley Crusher)
54         age 27
55     )
56 )";
57
58     var parser = new Parser();
59     var result = parser.Parse(input);
60
61     Assert.NotEmpty(result);
62     var formatted = result.Format();
63     Assert.Contains("empInfo", formatted);
64     Assert.Contains("employees:", formatted);
65     Assert.Contains("James Kirk", formatted);
66     Assert.Contains("Jean-Luc Picard", formatted);
67     Assert.Contains("Wesley Crusher", formatted);
68 }
69
70 [Fact]
71 public static void HeroExampleEquivalenceTest()
72 {
73     var version1 = @"empInfo
74 employees:
75 (
76     name (James Kirk)
77     age 40
78 )
79 (
80     name (Jean-Luc Picard)
81     age 45
82 )
83 (
84     name (Wesley Crusher)
85     age 27
86 )";
87
88     var version2 = @"empInfo
89 (
90     employees:
91     (
92         name (James Kirk)
93         age 40
94     )

```

```

95     (
96         name (Jean-Luc Picard)
97         age 45
98     )
99     (
100        name (Wesley Crusher)
101        age 27
102    )
103 ");
104
105    var parser = new Parser();
106    var result1 = parser.Parse(version1);
107    var result2 = parser.Parse(version2);
108
109    Assert.NotEmpty(result1);
110    Assert.NotEmpty(result2);
111
112    var formatted1 = result1.Format();
113    var formatted2 = result2.Format();
114
115    Assert.Equal(formatted1, formatted2);
116 }
117
118 [Fact]
119 public static void SetContextWithoutColonTest()
120 {
121     var input = @"empInfo
122 employees";
123
124     var parser = new Parser();
125     var result = parser.Parse(input);
126
127     Assert.NotEmpty(result);
128     var formatted = result.Format();
129     Assert.Contains("empInfo", formatted);
130     Assert.Contains("employees", formatted);
131 }
132
133 [Fact]
134 public static void SequenceContextWithColonTest()
135 {
136     var input = @"employees:
137 James Kirk
138 Jean-Luc Picard
139 Wesley Crusher";
140
141     var parser = new Parser();
142     var result = parser.Parse(input);
143
144     Assert.NotEmpty(result);
145     Assert.Single(result);
146     var formatted = result.Format();
147     Assert.Contains("employees:", formatted);
148     Assert.Contains("James Kirk", formatted);
149     Assert.Contains("Jean-Luc Picard", formatted);
150     Assert.Contains("Wesley Crusher", formatted);
151 }
152
153 [Fact]
154 public static void SequenceContextWithComplexValuesTest()
155 {
156     var input = @"employees:
157 (
158     name (James Kirk)
159     age 40
160 )
161 (
162     name (Jean-Luc Picard)
163     age 45
164 )";
165
166     var parser = new Parser();
167     var result = parser.Parse(input);
168
169     Assert.NotEmpty(result);
170     Assert.Single(result);
171     var formatted = result.Format();
172     Assert.Contains("employees:", formatted);
173     Assert.Contains("James Kirk", formatted);
174     Assert.Contains("Jean-Luc Picard", formatted);

```

```

175     }
176
177     [Fact]
178     public static void NestedSetAndSequenceContextsTest()
179     {
180         var input = @"company
181 departments:
182   engineering
183   sales
184 employees:
185   (name John)
186   (name Jane)";
187
188         var parser = new Parser();
189         var result = parser.Parse(input);
190
191         Assert.NotEmpty(result);
192         var formatted = result.Format();
193         Assert.Contains("company", formatted);
194         Assert.Contains("departments:", formatted);
195         Assert.Contains("employees:", formatted);
196     }
197
198     [Fact]
199     public static void DeeplyNestedMixedModesTest()
200     {
201         var input = @"root
202 level1
203   level2:
204     value1
205     value2
206   level2b
207     level3";
208
209         var parser = new Parser();
210         var result = parser.Parse(input);
211
212         Assert.NotEmpty(result);
213         var formatted = result.Format();
214         Assert.Contains("root", formatted);
215         Assert.Contains("level2:", formatted);
216     }
217 }
218

```

## 1.28 ./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class MultiQuoteParserTests
7      {
8          // Helper to extract single reference ID
9          private static string? GetSingleRefId(System.Collections.Generic.IList<Link<string>>
↪ result)
10         {
11             if (result.Count == 1 && result[0].Id == null && result[0].Values?.Count == 1)
12             {
13                 return result[0].Values![0].Id;
14             }
15             return result.Count == 1 ? result[0].Id : null;
16         }
17
18         // =====
19         // Backtick Quote Tests (Single Backtick)
20         // =====
21
22         [Fact]
23         public static void TestBacktickQuotedReference()
24         {
25             var parser = new Parser();
26             var result = parser.Parse("`backtick quoted`");
27             Assert.Equal("backtick quoted", GetSingleRefId(result));
28         }
29
30         [Fact]
31         public static void TestBacktickQuotedWithSpaces()
32         {
33             var parser = new Parser();

```

```

34     var result = parser.Parse("`text with spaces`");
35     Assert.Equal("text with spaces", GetSingleRefId(result));
36 }
37
38 [Fact]
39 public static void TestBacktickQuotedMultiline()
40 {
41     var parser = new Parser();
42     var result = parser.Parse("`line1\nline2`");
43     Assert.Single(result);
44     Assert.NotNull(result[0].Values);
45     Assert.Single(result[0].Values!);
46     Assert.Equal("line1\nline2", result[0].Values![0].Id);
47 }
48
49 [Fact]
50 public static void TestBacktickQuotedWithEscapedBacktick()
51 {
52     var parser = new Parser();
53     var result = parser.Parse("`text with `` escaped backtick`");
54     Assert.Equal("text with ` escaped backtick", GetSingleRefId(result));
55 }
56
57 // =====
58 // Single Quote Tests (with escaping)
59 // =====
60
61 [Fact]
62 public static void TestSingleQuoteWithEscapedSingleQuote()
63 {
64     var parser = new Parser();
65     var result = parser.Parse("'text with ' escaped quote'");
66     Assert.Equal("text with ' escaped quote", GetSingleRefId(result));
67 }
68
69 // =====
70 // Double Quote Tests (with escaping)
71 // =====
72
73 [Fact]
74 public static void TestDoubleQuoteWithEscapedDoubleQuote()
75 {
76     var parser = new Parser();
77     var result = parser.Parse("\"text with \" escaped quote\"");
78     Assert.Equal("text with \" escaped quote", GetSingleRefId(result));
79 }
80
81 // =====
82 // Double Quotes (2 quote chars) Tests
83 // =====
84
85 [Fact]
86 public static void TestDoubleDoubleQuotes()
87 {
88     var parser = new Parser();
89     var result = parser.Parse("\"\"double double quotes\"\"");
90     Assert.Equal("double double quotes", GetSingleRefId(result));
91 }
92
93 [Fact]
94 public static void TestDoubleDoubleQuotesWithSingleQuoteInside()
95 {
96     var parser = new Parser();
97     var result = parser.Parse("\"\"text with \" inside\"\"");
98     Assert.Equal("text with \" inside", GetSingleRefId(result));
99 }
100
101 [Fact]
102 public static void TestDoubleDoubleQuotesWithEscape()
103 {
104     var parser = new Parser();
105     var result = parser.Parse("\"\"text with \"\"\" escaped double\"\"");
106     Assert.Equal("text with \"\" escaped double", GetSingleRefId(result));
107 }
108
109 [Fact]
110 public static void TestDoubleSingleQuotes()
111 {
112     var parser = new Parser();

```

```

113     var result = parser.Parse("'double single quotes'");
114     Assert.Equal("double single quotes", GetSingleRefId(result));
115 }
116
117 [Fact]
118 public static void TestDoubleSingleQuotesWithSingleQuoteInside()
119 {
120     var parser = new Parser();
121     var result = parser.Parse("'text with ' inside'");
122     Assert.Equal("text with ' inside", GetSingleRefId(result));
123 }
124
125 [Fact]
126 public static void TestDoubleSingleQuotesWithEscape()
127 {
128     var parser = new Parser();
129     var result = parser.Parse("'text with ''' escaped single'");
130     Assert.Equal("text with ' escaped single", GetSingleRefId(result));
131 }
132
133 [Fact]
134 public static void TestDoubleBacktickQuotes()
135 {
136     var parser = new Parser();
137     var result = parser.Parse("`double backtick quotes`");
138     Assert.Equal("double backtick quotes", GetSingleRefId(result));
139 }
140
141 [Fact]
142 public static void TestDoubleBacktickQuotesWithBacktickInside()
143 {
144     var parser = new Parser();
145     var result = parser.Parse("`text with ` inside`");
146     Assert.Equal("text with ` inside", GetSingleRefId(result));
147 }
148
149 [Fact]
150 public static void TestDoubleBacktickQuotesWithEscape()
151 {
152     var parser = new Parser();
153     var result = parser.Parse("`text with ``` escaped backtick`");
154     Assert.Equal("text with `` escaped backtick", GetSingleRefId(result));
155 }
156
157 // =====
158 // Triple Quotes (3 quote chars) Tests
159 // =====
160
161 [Fact]
162 public static void TestTripleDoubleQuotes()
163 {
164     var parser = new Parser();
165     var result = parser.Parse("\"\"triple double quotes\"\"");
166     Assert.Equal("triple double quotes", GetSingleRefId(result));
167 }
168
169 [Fact]
170 public static void TestTripleDoubleQuotesWithDoubleQuoteInside()
171 {
172     var parser = new Parser();
173     var result = parser.Parse("\"\"text with \"\" inside\"\"");
174     Assert.Equal("text with \"\" inside", GetSingleRefId(result));
175 }
176
177 [Fact]
178 public static void TestTripleDoubleQuotesWithEscape()
179 {
180     var parser = new Parser();
181     var result = parser.Parse("\"\"text with \"\"\"\" escaped triple\"\"");
182     Assert.Equal("text with \"\"\"\" escaped triple", GetSingleRefId(result));
183 }
184
185 [Fact]
186 public static void TestTripleSingleQuotes()
187 {
188     var parser = new Parser();
189     var result = parser.Parse("'''triple single quotes'''");
190     Assert.Equal("triple single quotes", GetSingleRefId(result));

```

```

191     }
192
193     [Fact]
194     public static void TestTripleBacktickQuotes()
195     {
196         var parser = new Parser();
197         var result = parser.Parse("```triple backtick quotes```");
198         Assert.Equal("triple backtick quotes", GetSingleRefId(result));
199     }
200
201     // =====
202     // Quadruple Quotes (4 quote chars) Tests
203     // =====
204
205     [Fact]
206     public static void TestQuadrupleDoubleQuotes()
207     {
208         var parser = new Parser();
209         var result = parser.Parse("\"\"\"\"quadruple double quotes\"\"\"\"");
210         Assert.Equal("quadruple double quotes", GetSingleRefId(result));
211     }
212
213     [Fact]
214     public static void TestQuadrupleSingleQuotes()
215     {
216         var parser = new Parser();
217         var result = parser.Parse('\"'\"'quadruple single quotes'\"'\"');
218         Assert.Equal("quadruple single quotes", GetSingleRefId(result));
219     }
220
221     [Fact]
222     public static void TestQuadrupleBacktickQuotes()
223     {
224         var parser = new Parser();
225         var result = parser.Parse("````quadruple backtick quotes````");
226         Assert.Equal("quadruple backtick quotes", GetSingleRefId(result));
227     }
228
229     // =====
230     // Quintuple Quotes (5 quote chars) Tests
231     // =====
232
233     [Fact]
234     public static void TestQuintupleDoubleQuotes()
235     {
236         var parser = new Parser();
237         var result = parser.Parse("\"\"\"\"\"quintuple double quotes\"\"\"\"\"");
238         Assert.Equal("quintuple double quotes", GetSingleRefId(result));
239     }
240
241     [Fact]
242     public static void TestQuintupleSingleQuotes()
243     {
244         var parser = new Parser();
245         var result = parser.Parse('\"'\"'\"'quintuple single quotes'\"'\"'\"');
246         Assert.Equal("quintuple single quotes", GetSingleRefId(result));
247     }
248
249     [Fact]
250     public static void TestQuintupleBacktickQuotes()
251     {
252         var parser = new Parser();
253         var result = parser.Parse("``````quintuple backtick quotes``````");
254         Assert.Equal("quintuple backtick quotes", GetSingleRefId(result));
255     }
256
257     // =====
258     // Complex Scenarios Tests
259     // =====
260
261     [Fact]
262     public static void TestMixedQuotesInLink()
263     {
264         var parser = new Parser();
265         var result = parser.Parse("(\"double\" 'single' `backtick`)");
266         Assert.Single(result);
267         Assert.NotNull(result[0].Values);
268         Assert.Equal(3, result[0].Values!.Count);
269         Assert.Equal("double", result[0].Values![0].Id);

```

```

270     Assert.Equal("single", result[0].Values![1].Id);
271     Assert.Equal("backtick", result[0].Values![2].Id);
272 }
273
274 [Fact]
275 public static void TestBacktickAsIdInLink()
276 {
277     var parser = new Parser();
278     var result = parser.Parse("`myId` : value1 value2");
279     Assert.Single(result);
280     Assert.Equal("myId", result[0].Id);
281     Assert.NotNull(result[0].Values);
282     Assert.Equal(2, result[0].Values!.Count);
283 }
284
285 [Fact]
286 public static void TestCodeBlockLikeContent()
287 {
288     var parser = new Parser();
289     var result = parser.Parse("`const x = 1;`");
290     Assert.Equal("const x = 1;", GetSingleRefId(result));
291 }
292
293 [Fact]
294 public static void TestNestedQuotesInMarkdown()
295 {
296     var parser = new Parser();
297     var result = parser.Parse("`Use `code` in markdown`");
298     Assert.Equal("Use `code` in markdown", GetSingleRefId(result));
299 }
300
301 [Fact]
302 public static void TestJsonStringWithQuotes()
303 {
304     var parser = new Parser();
305     var result = parser.Parse("{\"key\": \"value\"}");
306     Assert.Equal("{ \"key\": \"value\"}", GetSingleRefId(result));
307 }
308
309 // =====
310 // Edge Cases
311 // =====
312
313 [Fact]
314 public static void TestWhitespacePreservedInQuotes()
315 {
316     var parser = new Parser();
317     var result = parser.Parse(" spaces ");
318     Assert.Equal(" spaces ", GetSingleRefId(result));
319 }
320
321 [Fact]
322 public static void TestMultilineInDoubleDoubleQuotes()
323 {
324     var parser = new Parser();
325     var result = parser.Parse("\\line1\\nline2\\");
326     Assert.Single(result);
327     Assert.NotNull(result[0].Values);
328     Assert.Single(result[0].Values!);
329     Assert.Equal("line1\\nline2", result[0].Values![0].Id);
330 }
331
332 // =====
333 // Unlimited Quotes (6+ quote chars) Tests
334 // =====
335
336 [Fact]
337 public static void TestUnlimitedQuotes6()
338 {
339     // Test 6-quote strings
340     var parser = new Parser();
341     var result = parser.Parse("\\\"\\\"\\\"\\\"hello\\\"\\\"\\\"\\\"");
342     Assert.Equal("hello", GetSingleRefId(result));
343 }
344
345 [Fact]
346 public static void TestUnlimitedQuotes10()
347 {

```

```

348         // Test 10-quote strings
349         var parser = new Parser();
350         var result = parser.Parse("\"\"\"\"\"\"\"\"\"\"\"\"\"\"very deeply
↪ quoted\"\"\"\"\"\"\"\"\"\"\"\"\"\"");
351         Assert.Equal("very deeply quoted", GetSingleRefId(result));
352     }
353
354     [Fact]
355     public static void TestUnlimitedQuotes6WithInnerQuotes()
356     {
357         // Test 6-quote strings with inner 5-quote sequences
358         var parser = new Parser();
359         var result = parser.Parse("\"\"\"\"\"\"\"\"\"\"hello with \"\"\"\"\"\" five quotes
↪ inside\"\"\"\"\"\"\"\"");
360         Assert.Equal("hello with \"\"\"\"\"\" five quotes inside", GetSingleRefId(result));
361     }
362
363     [Fact]
364     public static void TestUnlimitedSingleQuotes7()
365     {
366         // Test 7-quote single quote strings
367         var parser = new Parser();
368         var result = parser.Parse("''''''seven single quotes''''''");
369         Assert.Equal("seven single quotes", GetSingleRefId(result));
370     }
371
372     [Fact]
373     public static void TestUnlimitedBackticks8()
374     {
375         // Test 8-quote backtick strings
376         var parser = new Parser();
377         var result = parser.Parse("`````````eight backticks``````");
378         Assert.Equal("eight backticks", GetSingleRefId(result));
379     }
380 }
381 }

```

## 1.29 ./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class MultilineParserTests
7      {
8          [Fact]
9          public static void TwoLinksTest()
10         {
11             var source = @"(first: x y)
12 (second: a b)";
13             var parser = new Parser();
14             var links = parser.Parse(source);
15             var target = links.Format();
16             Assert.Equal(source, target);
17         }
18
19         [Fact]
20         public static void ParseAndStringifyTest()
21         {
22             var source = @"(papa (lovesMama: loves mama))
23 (son lovesMama)
24 (daughter lovesMama)
25 (all (love mama))";
26             var parser = new Parser();
27             var links = parser.Parse(source);
28             var target = links.Format();
29             Assert.Equal(source, target);
30         }
31
32         [Fact]
33         public static void ParseAndStringifyTest2()
34         {
35             var source = @"father (lovesMom: loves mom)
36 son lovesMom
37 daughter lovesMom
38 all (love mom)";
39             var links = new Parser().Parse(source);
40             var target = links.Format(lessParentheses: true);
41             Assert.Equal(source, target);

```

```

42     }
43
44     [Fact]
45     public static void ParseAndStringifyWithLessParenthesesTest()
46     {
47         var source = @"lovesMama: loves mama
48 papa lovesMama
49 son lovesMama
50 daughter lovesMama
51 all (love mama)";
52         var parser = new Parser();
53         var links = parser.Parse(source);
54         var target = links.Format(lessParentheses: true);
55         Assert.Equal(source, target);
56     }
57
58     [Fact]
59     public static void DuplicateIdentifiersTest()
60     {
61         var source = @"(a: a b)
62 (a: b c)";
63         var target = @"(a: a b)
64 (a: b c)";
65         var parser = new Parser();
66         var links = parser.Parse(source);
67         var formattedLinks = links.Format();
68         Assert.Equal(target, formattedLinks);
69     }
70
71     [Fact]
72     public static void ComplexStructureTest()
73     {
74         var input = @"(Type: Type Type)
75 Number
76 String
77 Array
78 Value
79 (property: name type)
80 (method: name params return)";
81
82         var result = new Parser().Parse(input);
83         Assert.NotEmpty(result);
84     }
85
86     [Fact]
87     public static void MixedFormatsTest()
88     {
89         // Mix of single-line and multi-line formats
90         var input = @"id1: value1
91 (id2: value2 value3)
92 simple ref
93 (complex:
94     nested1
95     nested2
96 )";
97
98         var result = new Parser().Parse(input);
99         Assert.NotEmpty(result);
100     }
101
102     [Fact]
103     public static void MultilineWithIdTest()
104     {
105         // Test multi-line link with id
106         var input = "(id: value1 value2)";
107         var result = new Parser().Parse(input);
108         Assert.NotEmpty(result);
109     }
110
111     [Fact]
112     public static void MultipleTopLevelElementsTest()
113     {
114         // Test multiple top-level elements
115         var input = "(elem1: val1)\n(elem2: val2)";
116         var result = new Parser().Parse(input);
117         Assert.NotEmpty(result);
118     }
119
120     [Fact]

```

```

121     public static void MultilineSimpleLinksTest()
122     {
123         var input = "(1: 1 1)\n(2: 2 2)";
124         var parser = new Parser();
125         var parsed = parser.Parse(input);
126         Assert.NotEmpty(parsed);
127
128         // Validate regular formatting
129         var output = parsed.Format();
130         Assert.Contains("(1: 1 1)", output);
131         Assert.Contains("(2: 2 2)", output);
132
133         // Validate alternate formatting matches input
134         var outputAlternate = parsed.Format();
135         Assert.Equal(input, outputAlternate);
136     }
137
138     [Fact]
139     public static void IndentedChildrenTest()
140     {
141         var input = "parent\n  child1\n  child2";
142         var parser = new Parser();
143         var parsed = parser.Parse(input);
144
145         // The parsed structure should have parent with children
146         Assert.NotEmpty(parsed);
147     }
148 }
149 }

```

### 1.30 ./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class MultilineQuotedStringTests
7      {
8          [Fact]
9          public static void TestMultilineDoubleQuotedReference()
10         {
11             var input = @"(
12     ""long
13 string literal representing
14 the reference""
15
16     'another
17 long string literal
18 as another reference'
19 )";
20             var parser = new Parser();
21             var result = parser.Parse(input);
22
23             Assert.NotEmpty(result);
24             Assert.Single(result);
25
26             var link = result[0];
27             Assert.Null(link.Id);
28             Assert.NotNull(link.Values);
29             Assert.Equal(2, link.Values.Count);
30
31             // The two references sit on separate lines inside the parentheses, so the
32             // nested context turns each of them into its own link.
33             Assert.Equal(@"long
34 string literal representing
35 the reference", link.Values![0].Values![0].Id);
36
37             Assert.Equal(@"another
38 long string literal
39 as another reference", link.Values![1].Values![0].Id);
40         }
41
42         [Fact]
43         public static void TestSimpleMultilineDoubleQuoted()
44         {
45             var input = @"(""line1
46 line2""");
47             var parser = new Parser();
48             var result = parser.Parse(input);
49

```

```

50     Assert.NotEmpty(result);
51     Assert.Single(result);
52
53     var link = result[0];
54     Assert.Null(link.Id);
55     Assert.NotNull(link.Values);
56     Assert.Single(link.Values);
57     Assert.Equal("line1\nline2", link.Values[0].Id);
58 }
59
60 [Fact]
61 public static void TestSimpleMultilineSingleQuoted()
62 {
63     var input = @"('line1
64 line2')";
65     var parser = new Parser();
66     var result = parser.Parse(input);
67
68     Assert.NotEmpty(result);
69     Assert.Single(result);
70
71     var link = result[0];
72     Assert.Null(link.Id);
73     Assert.NotNull(link.Values);
74     Assert.Single(link.Values);
75     Assert.Equal("line1\nline2", link.Values[0].Id);
76 }
77
78 [Fact]
79 public static void TestMultilineQuotedAsId()
80 {
81     var input = @"("multi
82 line
83 id": value1 value2)";
84     var parser = new Parser();
85     var result = parser.Parse(input);
86
87     Assert.NotEmpty(result);
88     Assert.Single(result);
89
90     var link = result[0];
91     Assert.Equal("multi\nline\nid", link.Id);
92     Assert.NotNull(link.Values);
93     Assert.Equal(2, link.Values.Count);
94 }
95 }
96 }

```

### 1.31 ./Link.Foundation.Links.Notation.Tests/NestedIndentationTests.cs

```

1  using Xunit;
2
3  namespace Link.Foundation.Links.Notation.Tests
4  {
5      /// <summary>
6      /// Tests for indentation inside parentheses.
7      ///
8      /// See https://github.com/link-foundation/links-notation/issues/282
9      ///
10     /// Indentation is structural at the root, so it must be structural inside parentheses too:
11     /// a parenthesized group opens a nested context that starts fresh at indentation level zero
12     /// and follows exactly the root's rules.
13     /// </summary>
14     public class NestedIndentationTests
15     {
16         private static string FormatSource(string source) => new Parser().Parse(source).Format();
17
18         private static void AssertFormat(string source, string expected) =>
19         ↪ Assert.Equal(expected, FormatSource(source));
20
21         [Fact]
22         public void ParenthesesReproduceRootIndentation()
23         {
24             AssertFormat("a\n b\nc\n d", "(a)\n((a) (b))\n(c)\n((c) (d))");
25
26             // The same lines inside parentheses keep the same structure, nested under
27             // the link the group belongs to.
28             AssertFormat("array (\n a\n b\n c\n d\n)", "(array ((a) ((a) (b)) (c) ((c)
29 ↪ (d))))");
30         }
31     }

```

```

29
30 [Fact]
31 public void ParenthesesKeepRecordBoundaries()
32 {
33     var source = "value (\n id \"1\"\n label \"one\"\n)";
34     Assert.Format(source, "(value ((id 1) (label one)))");
35
36     var links = new Parser().Parse(source);
37     Assert.Single(links);
38
39     var group = links[0].Values![1];
40     Assert.Null(group.Id);
41     Assert.Equal(2, group.Values!.Count);
42     Assert.Equal("id", group.Values[0].Values![0].Id);
43     Assert.Equal("1", group.Values[0].Values![1].Id);
44     Assert.Equal("label", group.Values[1].Values![0].Id);
45     Assert.Equal("one", group.Values[1].Values![1].Id);
46 }
47
48 [Fact]
49 public void ParenthesesKeepSeveralRecordsSeparate()
50 {
51     Assert.Format(
52         "value (\n (id \"1\" label \"one\")\n (id \"2\" label \"two\")\n)",
53         "(value ((id 1 label one) (id 2 label two)))");
54 }
55
56 [Fact]
57 public void ParenthesesNestDeeply()
58 {
59     Assert.Format("outer (\n inner (\n x 1\n y 2\n )\n z 3\n)", "(outer ((inner
↪ ((x 1) (y 2))) (z 3)))");
60 }
61
62 [Fact]
63 public void SingleLineParenthesesAreUnchanged()
64 {
65     Assert.Format("(a b c)", "(a b c)");
66     Assert.Format("(1: 2 3)", "(1: 2 3)");
67     Assert.Format("(a: b c)", "(a: b c)");
68     Assert.Format("((a b))", "((a b))");
69     Assert.Format("(a)", "(a)");
70     Assert.Format("()", "()");
71 }
72
73 [Fact]
74 public void ParenthesesWithIndentedIdSyntax()
75 {
76     Assert.Format("(\\n a:\\n b\\n c\\n)", "(a: b c)");
77 }
78
79 [Fact]
80 public void EmployeeRecordsKeepTheirFields()
81 {
82     var source = "empInfo\\n employees:\\n (\\n name (James Kirk)\\n age 40\\n
↪ )\\n"
83         + " (\\n name (Jean-Luc Picard)\\n age 45\\n )";
84     var expected = "(empInfo\\n((empInfo) (employees: ((name (James Kirk)) (age 40)))"
85         + " ((name (Jean-Luc Picard)) (age 45))))";
86     Assert.Format(source, expected);
87 }
88 }
89 }

```

### 1.32 ./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs

```

1 using System;
2 using System.Linq;
3 using Xunit;
4
5 namespace Link.Foundation.Links.Notation.Tests
6 {
7     public static class NestedParserTests
8     {
9         [Fact]
10         public static void SignificantWhitespaceTest()
11         {
12             var source = @"
13 users
14     user1

```

```

15         id
16         43
17         name
18         first
19             John
20         last
21             Williams
22         location
23             New York
24         age
25             23
26     user2
27         id
28         56
29         name
30         first
31             Igor
32         middle
33             Petrovich
34         last
35             Ivanov
36         location
37             Moscow
38         age
39             20";
40         var target = @"(users)
41 ((users) (user1))
42 (((users) (user1)) (id))
43 (((users) (user1)) (id)) (43))
44 (((users) (user1)) (name))
45 (((users) (user1)) (name)) (first))
46 (((users) (user1)) (name)) (first)) (John))
47 (((users) (user1)) (name)) (last))
48 (((users) (user1)) (name)) (last)) (Williams))
49 ((users) (user1)) (location))
50 (((users) (user1)) (location)) (New York))
51 ((users) (user1)) (age))
52 (((users) (user1)) (age)) (23))
53 ((users) (user2))
54 (((users) (user2)) (id))
55 (((users) (user2)) (id)) (56))
56 (((users) (user2)) (name))
57 (((users) (user2)) (name)) (first))
58 (((users) (user2)) (name)) (first)) (Igor))
59 (((users) (user2)) (name)) (middle))
60 (((users) (user2)) (name)) (middle)) (Petrovich))
61 (((users) (user2)) (name)) (last))
62 (((users) (user2)) (name)) (last)) (Ivanov))
63 ((users) (user2)) (location))
64 (((users) (user2)) (location)) (Moscow))
65 ((users) (user2)) (age))
66 (((users) (user2)) (age)) (20))";
67         var parser = new Parser();
68         var links = parser.Parse(source);
69         var formattedLinks = links.Format();
70         Assert.Equal(target, formattedLinks);
71     }
72
73     [Fact]
74     public static void SimpleSignificantWhitespaceTest()
75     {
76         var source = @"a
77 b
78 c";
79         var target = @"(a)
80 (b)
81 (c)";
82         var parser = new Parser();
83         var links = parser.Parse(source);
84         var formattedLinks = links.Format();
85         Assert.Equal(target, formattedLinks);
86     }
87
88     [Fact]
89     public static void TwoSpacesSizedWhitespaceTest()
90     {
91         var source = @"
92 users
93     user1";
94         var target = @"(users)

```

```

95 ((users) (user1));
96     var parser = new Parser();
97     var links = parser.Parse(source);
98     var formattedLinks = links.Format();
99     Assert.Equal(target, formattedLinks);
100 }
101
102 [Fact]
103 public static void ParseNestedStructureWithIndentationTest()
104 {
105     var source = @"parent
106 child1
107 child2";
108     var parser = new Parser();
109     var links = parser.Parse(source);
110     Assert.NotNull(links);
111     // Should create 3 links: (parent), (parent child1), (parent child2)
112     Assert.Equal(3, links.Count);
113 }
114
115 [Fact]
116 public static void IndentationConsistencyTest()
117 {
118     // Test that indentation must be consistent
119     var input = @"parent
120 child1
121 child2"; // Inconsistent indentation
122     var result = new Parser().Parse(input);
123     // This should parse but child2 won't be a child of parent due to different
124     // indentation
125     Assert.NotEmpty(result);
126 }
127
128 [Fact]
129 public static void IndentationBasedChildrenTest()
130 {
131     var input = @"parent
132 child1
133 child2
134 grandchild";
135     var parser = new Parser();
136     var result = parser.Parse(input);
137     Assert.NotNull(result);
138     // Expected flattened links:
139     // (parent), (parent child1), (parent child2), ((parent child2) grandchild)
140     Assert.Equal(4, result.Count);
141 }
142
143 [Fact]
144 public static void ComplexIndentationTest()
145 {
146     var input = @"root
147 level1a
148   level2a
149   level2b
150 level1b
151   level2c";
152     var parser = new Parser();
153     var result = parser.Parse(input);
154     Assert.NotNull(result);
155     // Expected flattened links:
156     // (root), (root level1a), ((root level1a) level2a), ((root level1a) level2b),
157     // (root level1b), ((root level1b) level2c)
158     Assert.Equal(6, result.Count);
159 }
160
161 [Fact]
162 public static void NestedLinksTest()
163 {
164     var input = "(1: (2: (3: 3)))";
165     var parser = new Parser();
166     var parsed = parser.Parse(input);
167     Assert.NotEmpty(parsed);
168
169     // Validate regular formatting
170     var output = parsed.Format();
171     Assert.NotEmpty(output);
172
173     // Validate that the structure is properly nested

```

```

173     Assert.Single(parsed);
174 }
175
176 [Fact]
177 public static void IndentationParserTest()
178 {
179     var input = "parent\n  child1\n  child2";
180     var parser = new Parser();
181     var result = parser.Parse(input);
182     Assert.NotEmpty(result);
183     // Should have parent link
184     var parentLink = result.FirstOrDefault(l => l.Id == "parent");
185 }
186
187 [Fact]
188 public static void NestedIndentationParserTest()
189 {
190     var input = "parent\n  child\n    grandchild";
191     var parser = new Parser();
192     var result = parser.Parse(input);
193     Assert.NotEmpty(result);
194     // Should create nested structure with proper hierarchy
195     Assert.True(result.Count >= 1);
196 }
197
198 [Fact]
199 public static void ThreeLevelNestingRoundtripTest()
200 {
201     var input = "(1: (2: (3: 3)))";
202     var parser = new Parser();
203     var parsed = parser.Parse(input);
204
205     // Validate round-trip
206     var output = parsed.Format();
207     Assert.Equal(input, output);
208 }
209
210 [Fact]
211 public static void DeepNestedStructureRoundtripTest()
212 {
213     var input = "(a: (b: (c: (d: d))))";
214     var parser = new Parser();
215     var parsed = parser.Parse(input);
216
217     // Validate round-trip
218     var output = parsed.Format();
219     Assert.Equal(input, output);
220 }
221
222 [Fact]
223 public static void MultipleNestedLinksRoundtripTest()
224 {
225     var input = "(parent: (child1: value1) (child2: value2))";
226     var parser = new Parser();
227     var parsed = parser.Parse(input);
228
229     // Validate round-trip
230     var output = parsed.Format();
231     Assert.Equal(input, output);
232 }
233 }
234 }

```

### 1.33 ./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class NestedSelfReferenceTests
7      {
8          [Fact]
9          public static void NestedSelfReferencedObjectInPairValueTest()
10         {
11             // Test case from PARSER BUG.md
12             // This should parse a dict with two pairs, where the second pair's value
13             // is itself a self-referenced dict definition (obj 1: dict ...)
14             var notation = "(obj 0: dict ((str bmFtZQ==) (str ZGljdDE=)) ((str b3RoZXI=) (obj 1:
↵ dict ((str bmFtZQ==) (str ZGljdDI=)) ((str b3RoZXI=) obj 0))))";

```

```

15
16 var parser = new Parser();
17 var links = parser.Parse(notation);
18
19 // Should parse exactly one top-level link
20 Assert.Single(links);
21
22 var link = links[0];
23
24 // Top-level link should have ID "obj 0"
25 Assert.Equal("obj 0", link.Id);
26
27 // Should have: type marker + 2 pairs = 3 values
28 Assert.Equal(3, link.Values!.Count);
29
30 // First value is the type marker "dict"
31 Assert.Equal("dict", link.Values[0].Id);
32
33 // Second and third values are the two pairs
34 var pair1 = link.Values[1];
35 var pair2 = link.Values[2];
36
37 // Pair 1: ((str bmFtZQ==) (str ZGldDI=))
38 // This is a parenthesized expression containing two sub-expressions
39 Assert.Null(pair1.Id);
40 Assert.Equal(2, pair1.Values!.Count);
41
42 // First element of pair1: (str bmFtZQ==)
43 Assert.Null(pair1.Values[0].Id);
44 Assert.Equal(2, pair1.Values![0].Values!.Count);
45 Assert.Equal("str", pair1.Values![0].Values![0].Id);
46 Assert.Equal("bmFtZQ==", pair1.Values![0].Values![1].Id);
47
48 // Second element of pair1: (str ZGldDI=)
49 Assert.Null(pair1.Values[1].Id);
50 Assert.Equal(2, pair1.Values![1].Values!.Count);
51 Assert.Equal("str", pair1.Values![1].Values![0].Id);
52 Assert.Equal("ZGldDI=", pair1.Values![1].Values![1].Id);
53
54 // Pair 2: ((str b3RoZXI=) (obj 1: dict ...))
55 // This is a parenthesized expression containing two sub-expressions
56 Assert.Null(pair2.Id);
57 Assert.Equal(2, pair2.Values!.Count);
58
59 // First element of pair2: (str b3RoZXI=)
60 Assert.Null(pair2.Values[0].Id);
61 Assert.Equal(2, pair2.Values![0].Values!.Count);
62 Assert.Equal("str", pair2.Values![0].Values![0].Id);
63 Assert.Equal("b3RoZXI=", pair2.Values![0].Values![1].Id);
64
65 // Second element of pair2: (obj 1: dict ((str bmFtZQ==) (str ZGldDI=)) ((str
↪ b3RoZXI=) obj 0))
66 // THIS IS THE KEY TEST - obj 1 should have its ID preserved
67 var obj1 = pair2.Values[1];
68 Assert.Equal("obj 1", obj1.Id);
69 Assert.NotNull(obj1.Values);
70 Assert.Equal(3, obj1.Values.Count); // type marker + 2 pairs
71
72 // obj 1's type marker
73 Assert.Equal("dict", obj1.Values[0].Id);
74
75 // obj 1's first pair: ((str bmFtZQ==) (str ZGldDI=))
76 var obj1Pair1 = obj1.Values[1];
77 Assert.Equal(2, obj1Pair1.Values!.Count);
78 Assert.Null(obj1Pair1.Values[0].Id);
79 Assert.Equal(2, obj1Pair1.Values![0].Values!.Count);
80 Assert.Equal("str", obj1Pair1.Values![0].Values![0].Id);
81 Assert.Equal("bmFtZQ==", obj1Pair1.Values![0].Values![1].Id);
82 Assert.Null(obj1Pair1.Values[1].Id);
83 Assert.Equal(2, obj1Pair1.Values![1].Values!.Count);
84 Assert.Equal("str", obj1Pair1.Values![1].Values![0].Id);
85 Assert.Equal("ZGldDI=", obj1Pair1.Values![1].Values![1].Id);
86
87 // obj 1's second pair: ((str b3RoZXI=) obj 0) - reference back to obj 0
88 var obj1Pair2 = obj1.Values[2];
89 Assert.Equal(2, obj1Pair2.Values!.Count);
90 Assert.Null(obj1Pair2.Values[0].Id);
91 Assert.Equal(2, obj1Pair2.Values![0].Values!.Count);
92 Assert.Equal("str", obj1Pair2.Values![0].Values![0].Id);

```

```

93     Assert.Equal("b3RoZXI=", obj1Pair2.Values![0].Values![1].Id);
94     Assert.Equal("obj 0", obj1Pair2.Values[1].Id);
95     Assert.Null(obj1Pair2.Values[1].Values); // Just a reference, no nested values
96 }
97
98 [Fact]
99 public static void SelfReferenceAsDirectChildWorksCorrectlyTest()
100 {
101     // This pattern should work (and did work before)
102     var notation = "(obj 0: list (int 1) (int 2) (obj 1: list (int 3) (int 4) obj 0))";
103
104     var parser = new Parser();
105     var links = parser.Parse(notation);
106
107     Assert.Single(links);
108     Assert.Equal("obj 0", links[0].Id);
109     Assert.Equal(4, links[0].Values!.Count); // list + 1 + 2 + obj 1
110
111     // The fourth value should be obj 1 with a self-reference
112     var obj1 = links[0].Values![3];
113     Assert.Equal("obj 1", obj1.Id);
114     Assert.Equal(4, obj1.Values!.Count); // list + 3 + 4 + obj 0
115     Assert.Equal("obj 0", obj1.Values[3].Id); // Reference to obj 0
116 }
117 }
118 }

```

### 1.34 ./Link.Foundation.Links.Notation.Tests/NestingDepthTests.cs

```

1  using System;
2  using System.Linq;
3  using System.Threading;
4  using Xunit;
5
6  namespace Link.Foundation.Links.Notation.Tests
7  {
8      /// <summary>
9      /// Reading nested groups must take time that grows with the size of the document,
10     /// not with two to the power of its nesting depth
11     /// (<see href="https://github.com/link-foundation/links-notation/issues/314">#314</see>).
12     /// Before the fix, every level of nesting doubled the work: a document nested six
13     /// levels deep took seconds. Each case runs on its own thread and fails when it has
14     /// not finished within the budget, so a regression fails instead of hanging.
15     /// </summary>
16     public class NestingDepthTests
17     {
18         /// Generous enough for a debug build on a slow machine, and far below what the
19         /// exponential reading took at the depths used here.
20         private static readonly TimeSpan Budget = TimeSpan.FromSeconds(5);
21
22         /// The depth used for the shapes that used to take exponential time.
23         private const int Deep = 32;
24
25         /// A depth that shows the reading stays linear well past the shallow cases.
26         private const int VeryDeep = 256;
27
28         private static string Closed(int depth) => new string('(', depth) + "a" + new
↪ string(')', depth);
29
30         private static string ValueAfter(int depth) => new string('(', depth) + "a" +
↪ string.Concat(Enumerable.Repeat(" b", depth));
31
32         private static string Unclosed(int depth) => new string('(', depth) + "a";
33
34         /// Unclosed groups on lines that are each indented one space deeper.
35         private static string IndentedUnclosed(int depth) =>
36         ↪ string.Join("\n", Enumerable.Range(0, depth).Select(level => new string(' ', level)
↪ + "(a)"));
37
38         /// Formats the parsed document, or names the exception parsing raised.
39         private static string ReadWithinBudget(string what, string source)
40         {
41             string? outcome = null;
42             var reader = new Thread(() =>
43             {
44                 try
45                 {
46                     // The documents nest deeper than the default limit on purpose; only the
47                     // time they take is measured here.
48                     outcome = new Parser(true, int.MaxValue).Parse(source).Format();

```

```

49         }
50         catch (Exception error)
51         {
52             outcome = error.GetType().Name;
53         }
54     }, 512 * 1024 * 1024)
55     {
56         // A reader that runs past the budget must not keep the test run alive.
57         IsBackground = true,
58     };
59     reader.Start();
60     Assert.True(reader.Join(Budget), $"{what} did not finish within {Budget}");
61     return outcome!;
62 }
63
64 [Fact]
65 public void ClosedGroupsReadInLinearTime()
66 {
67     var source = Closed(Deep);
68     Assert.Equal(source, ReadWithinBudget("closed groups", source));
69 }
70
71 [Fact]
72 public void ValuesAfterGroupsReadInLinearTime()
73 {
74     var source = ValueAfter(Deep);
75     Assert.Equal($"({source})", ReadWithinBudget("values after groups", source));
76 }
77
78 [Fact]
79 public void UnclosedGroupsFailInLinearTime()
80 {
81     Assert.Equal(nameof(ParseException), ReadWithinBudget("unclosed groups",
↵ Unclosed(Deep)));
82 }
83
84 [Fact]
85 public void UnclosedGroupsOnIndentedLinesFailInLinearTime()
86 {
87     Assert.Equal(nameof(ParseException), ReadWithinBudget("unclosed indented groups",
↵ IndentedUnclosed(Deep)));
88 }
89
90 [Fact]
91 public void ValuesAfterVeryDeepGroupsReadInLinearTime()
92 {
93     var source = ValueAfter(VeryDeep);
94     Assert.Equal($"({source})", ReadWithinBudget("very deep values after groups",
↵ source));
95 }
96
97 [Fact]
98 public void GroupFollowedByValuesKeepsItsStructure()
99 {
100     var cases = new[]
101     {
102         new[] { "(a) b", "((a) b)" },
103         new[] { "(a) (b) c", "((a) (b) c)" },
104         new[] { "((a) b) c", "(((a) b) c)" },
105         new[] { "(a: b) c", "((a: b) c)" },
106         new[] { "(a)\n(b) c", "(a)\n((b) c)" },
107         new[] { "x\n (a) b\n (c)", "(x)\n((x) ((a) b))\n((x) (c))" },
108     };
109
110     var parser = new Parser();
111     foreach (var testCase in cases)
112     {
113         Assert.Equal(testCase[1], parser.Parse(testCase[0]).Format());
114     }
115 }
116 }
117 }

```

### 1.35 ./Link.Foundation.Links.Notation.Tests/NestingLimitTests.cs

```

1 using System;
2 using System.Linq;
3 using System.Threading;
4 using Xunit;
5

```

```

6 namespace Link.Foundation.Links.Notation.Tests
7 {
8     /// <summary>
9     /// Links nested too deeply are refused with an error rather than recursed into until
10    /// the stack overflows, which ends the process where no catch block sees it
11    /// (https://github.com/link-foundation/links-notation/issues/315).
12    /// </summary>
13    /// <remarks>
14    /// Every parenthesized group and every indentation level is one level, and the lines of
15    /// a document start at level 0. The positions asserted here are the ones the Rust and
16    /// JavaScript ports report for the same input.
17    /// </remarks>
18    public static class NestingLimitTests
19    {
20        private static string Parens(int depth) => new string('(', depth) + "a" + new
↪ string(')', depth);
21
22        private static string Values(int depth) =>
23            string.Concat(Enumerable.Repeat("a ", depth)) + "b" + new string(' ', depth);
24
25        private static string Indentation(int depth) =>
26            string.Concat(Enumerable.Range(0, depth + 1).Select(level => new string(' ', level)
↪ + "a\n"));
27
28        private static ParseException TooDeep(string document, int maxDepth)
29        {
30            var error = Assert.Throws<ParseException>(() => new Parser(true,
↪ maxDepth).Parse(document));
31            Assert.Equal(maxDepth, error.MaxDepth);
32            return error;
33        }
34
35        private static bool Accepted(string document, int maxDepth) =>
36            new Parser(true, maxDepth).Parse(document).Count > 0;
37
38        [Fact]
39        public static void DefaultLimitIsSharedByEveryImplementationTest()
40        {
41            Assert.Equal(64, Parser.DefaultMaxDepth);
42            Assert.Equal(Parser.DefaultMaxDepth, new Parser().MaxDepth);
43            Assert.Equal(Parser.DefaultMaxDepth, new Parser(comments: false).MaxDepth);
44            Assert.Equal(3, new Parser { MaxDepth = 3 }.MaxDepth);
45        }
46
47        [Fact]
48        public static void NegativeLimitIsRejectedTest()
49        {
50            Assert.Throws<ArgumentOutOfRangeException>(() => new Parser(true, -1));
51            Assert.Throws<ArgumentOutOfRangeException>(() => new Parser { MaxDepth = -1 });
52        }
53
54        [Fact]
55        public static void ParenthesesUpToTheLimitAreAcceptedTest()
56        {
57            Assert.True(Accepted(Parens(3), 3));
58            Assert.True(Accepted(Values(3), 3));
59            Assert.NotEmpty(new Parser().Parse(Parens(Parser.DefaultMaxDepth)));
60            Assert.NotEmpty(new Parser().Parse(Values(Parser.DefaultMaxDepth)));
61        }
62
63        [Fact]
64        public static void EveryGroupAndEveryIndentationLevelIsOneLevelTest()
65        {
66            Assert.True(Accepted("(a)", 1));
67            Assert.True(Accepted("a b c", 0));
68            TooDeep("(a)", 0);
69            TooDeep("((a))", 1);
70            TooDeep("(a (b))", 1);
71            TooDeep("a\n (b)\n", 1);
72        }
73
74        [Fact]
75        public static void ParenthesesPastTheLimitAreRefusedAtTheGroupThatIsTooDeepTest()
76        {
77            var error = TooDeep(Parens(4), 3);
78
79            Assert.Equal((1, 4, 3), (error.Line, error.Column, error.Offset));

```

```

80         Assert.Equal("line 1, column 4: nesting depth exceeds the maximum of 3",
↪     error.Summary);
81         Assert.Equal("1 | (((a)))\n | ^", error.Snippet);
82         Assert.Equal(
83             ↪ + "Nesting too deep at line 1, column 4: nesting depth exceeds the maximum of 3\n"
84                 "1 | (((a)))\n" +
85                 " | ^",
86             error.Message);
87     }
88
89     [Fact]
90     public static void GroupsInValuePositionCountLikeAnyOtherGroupTest()
91     {
92         var error = TooDeep(Values(4), 3);
93
94         Assert.Equal((1, 10), (error.Line, error.Column));
95     }
96
97     [Fact]
98     public static void IndentationUpToTheLimitIsAcceptedTest()
99     {
100         Assert.True(Accepted(Indentation(3), 3));
101         Assert.NotEmpty(new Parser().Parse(Indentation(Parser.DefaultMaxDepth)));
102     }
103
104     [Fact]
105     public static void IndentationPastTheLimitIsRefusedAtTheLineThatIsTooDeepTest()
106     {
107         var error = TooDeep(Indentation(4), 3);
108
109         Assert.Equal((5, 5), (error.Line, error.Column));
110         Assert.Equal(" a", error.LineText);
111     }
112
113     [Fact]
114     public static void GroupsAndIndentationAddUpTest()
115     {
116         // `(b)` on the line indented once is at level 2.
117         Assert.True(Accepted("a\n (b)\n", 2));
118         var error = TooDeep("a\n (b)\n", 1);
119
120         Assert.Equal((2, 3), (error.Line, error.Column));
121     }
122
123     [Fact]
124     public static void IndentationInsideAGroupCountsOnTopOfTheGroupTest()
125     {
126         // `b` is inside one group and indented once within it.
127         Assert.True(Accepted("(a\n b)\n", 2));
128         var error = TooDeep("(a\n b)\n", 1);
129
130         Assert.Equal((2, 3), (error.Line, error.Column));
131     }
132
133     [Fact]
134     public static void TrailingSpacesOnADeepLineAreNotADeeperLineTest()
135     {
136         Assert.True(Accepted("a\n b\n c \n", 2));
137         Assert.True(Accepted("a\n b\n c\n ", 2));
138     }
139
140     [Fact]
141     public static void SyntaxErrorIsNotMistakenForNestingThatIsTooDeepTest()
142     {
143         var error = Assert.Throws<ParseException>(() => new Parser().Parse("a: b: c"));
144
145         Assert.Null(error.MaxDepth);
146         Assert.StartsWith("Syntax error at ", error.Message);
147     }
148
149     [Fact]
150     public static void RefusesADocumentFarPastTheLimitWithoutOverflowingTheStackTest()
151     {
152         // Before the limit existed each of these overflowed the stack, which ends the
153         // process. The documents are parsed on a 1 MiB stack, smaller than any thread
154         // .NET starts by default.
155         string?[] messages = new string?[3];
156         Exception? unexpected = null;

```

```

157     var thread = new Thread(() =>
158     {
159         try
160         {
161             var documents = new[] { Parens(100 000), Values(100 000), Indentation(2 000)
↪ };
162             for (var index = 0; index < documents.Length; index++)
163             {
164                 var error = Assert.Throws<ParseException>(() => new
↪ Parser().Parse(documents[index]));
165                 Assert.Equal(Parser.DefaultMaxDepth, error.MaxDepth);
166                 messages[index] = error.Message;
167             }
168         }
169         catch (Exception error)
170         {
171             unexpected = error;
172         }
173     }, 1 << 20);
174     thread.Start();
175     thread.Join();
176
177     Assert.Null(unexpected);
178     Assert.All(messages, message => Assert.StartsWith("Nesting too deep at ", message));
179 }
180
181 [Fact]
182 public static void StreamParserReportsWhereTheNestingIsTooDeepTest()
183 {
184     var stream = new StreamParser(new Parser(true, 1));
185     stream.Write("a\\nb ((c))\\n");
186
187     var error = Assert.Throws<StreamParseException>(() => stream.Finish());
188
189     Assert.Equal(1, error.ParseError.MaxDepth);
190     Assert.Equal((2, 4, 5), (error.Line, error.Column, error.Offset));
191 }
192 }
193 }

```

### 1.36 ./Link.Foundation.Links.Notation.Tests/ParseErrorPositionTests.cs

```

1 using System;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     /// <summary>
7     /// A parse error has to say where the document stopped making sense.
8     /// </summary>
9     /// <remarks>
10    /// The positions asserted here are the ones the JavaScript and Rust ports report for
11    /// the same input, so the implementations can be held to the same contract
12    /// (https://github.com/link-foundation/links-notation/issues/302).
13    /// </remarks>
14    public static class ParseErrorPositionTests
15    {
16        private static ParseException SyntaxError(string document) =>
17            Assert.Throws<ParseException>(() => new Parser().Parse(document));
18
19        [Fact]
20        public static void ReportsTheLineAndColumnOfTheDefectTest()
21        {
22            // The example from the issue: the defect is the second colon on line 2,
23            // and the two lines after it are fine.
24            var error = SyntaxError("ci gate x\\nstage: rust: nexttest\\nnext stage\\n clippy");
25
26            Assert.Equal(2, error.Line);
27            Assert.Equal(12, error.Column);
28            Assert.Equal(':', error.Found);
29        }
30
31        [Fact]
32        public static void OffsetAgreesWithTheOtherImplementationsTest()
33        {
34            // JavaScript and Rust report offset 21, line 2, column 12 for this document.
35            var error = SyntaxError("ci gate x\\nstage: rust: nexttest\\n");
36
37            Assert.Equal(21, error.Offset);
38            Assert.Equal(2, error.Line);

```

```

39     Assert.Equal(12, error.Column);
40 }
41
42 [Fact]
43 public static void ReportsTheLineALateDefectIsOnTest()
44 {
45     var error = SyntaxError("a\nb\nc\nd\ne: f: g\nh\n");
46
47     Assert.Equal(5, error.Line);
48     Assert.Equal(5, error.Column);
49     Assert.Equal("e: f: g", error.LineText);
50 }
51
52 [Fact]
53 public static void ReportsTheEndOfTheDocumentWhenAGroupIsNeverClosedTest()
54 {
55     var error = SyntaxError("a (b\n");
56
57     Assert.Equal(5, error.Offset);
58     Assert.Equal(2, error.Line);
59     Assert.Equal(1, error.Column);
60     Assert.Null(error.Found);
61     Assert.Contains("end of input", error.Message);
62 }
63
64 [Fact]
65 public static void ReportsAnUnmatchedClosingParenthesisTest()
66 {
67     var error = SyntaxError("a b)\n");
68
69     Assert.Equal(3, error.Offset);
70     Assert.Equal(1, error.Line);
71     Assert.Equal(4, error.Column);
72     Assert.Equal(')', error.Found);
73 }
74
75 [Fact]
76 public static void MessagePointsACaretAtTheOffendingCharacterTest()
77 {
78     var error = SyntaxError("ci gate x\nstage: rust: nextest\n");
79
80     Assert.Equal("line 2, column 12: unexpected \":\"", error.Summary);
81     Assert.Equal("2 | stage: rust: nextest\n |", error.Snippet);
82     Assert.Equal(
83         "Syntax error at line 2, column 12: unexpected \":\"\n2 | stage: rust: nextest\n",
84         error.Message);
85 }
86
87 [Fact]
88 public static void MessageQuotesOneLineRatherThanTheRestOfTheDocumentTest()
89 {
90     var document = "ci gate x\nstage: rust: nextest\n"
91         + string.Concat(System.Linq.Enumerable.Repeat("trailing line\n", 500));
92
93     var error = SyntaxError(document);
94
95     Assert.Contains("line 2, column 12", error.Message);
96     Assert.DoesNotContain("trailing line", error.Message);
97     Assert.True(error.Message.Length < 200, $"message is {error.Message.Length}
98     characters");
99 }
100
101 [Fact]
102 public static void MessageOfALongLineStaysAMessageTest()
103 {
104     var document = new string('a', 400) + ": " + new string('b', 400) + ": c";
105
106     var error = SyntaxError(document);
107
108     Assert.Equal(1, error.Line);
109     Assert.Equal(803, error.Column);
110     Assert.Contains("...", error.Message);
111     Assert.True(error.Message.Length < 300, $"message is {error.Message.Length}
112     characters");
113 }
114
115 [Fact]
116 public static void KeepsCatchingCodeThatExpectsAFormatExceptionWorkingTest()

```

```

115     {
116         // The generated parser used to raise a bare FormatException; code that catches
117         // it keeps working, and now reads a position off the exception it catches.
118         var error = Assert.Throws<ParseException>(() => new Parser().Parse("a: b: c"));
119
120         Assert.IsAssignableFrom<FormatException>(error);
121         Assert.Equal(4, error.Offset);
122     }
123
124     [Fact]
125     public static void DoesNotMentionTheGrammarInternalsTest()
126     {
127         var error = SyntaxError("a: b: c");
128
129         Assert.DoesNotContain("Failed to parse", error.Message);
130         Assert.DoesNotContain("document", error.Message);
131     }
132
133     [Fact]
134     public static void ADocumentThatParsesReportsNothingTest()
135     {
136         var links = new Parser().Parse("a: b\n c: d\n");
137
138         Assert.Equal(2, links.Count);
139     }
140 }
141 }

```

### 1.37 ./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class SingleLineParserTests
7      {
8          [Fact]
9          public static void SingleLinkTest()
10         {
11             var source = @"(address: source target)";
12             var parser = new Parser();
13             var links = parser.Parse(source);
14             var target = links.Format();
15             Assert.Equal(source, target);
16         }
17
18         [Fact]
19         public static void TripletSingleLinkTest()
20         {
21             var source = @"(papa has car)";
22             var parser = new Parser();
23             var links = parser.Parse(source);
24             var target = links.Format();
25             Assert.Equal(source, target);
26         }
27
28         [Fact]
29         public static void BugTest1()
30         {
31             var source = @"(ignore conan-center-index repository)";
32             var links = new Parser().Parse(source);
33             var target = links.Format();
34             Assert.Equal(source, target);
35         }
36
37         [Fact]
38         public static void QuotedReferencesTest()
39         {
40             var source = @"(a: 'b' "c")";
41             var target = @"(a: b c)";
42             var parser = new Parser();
43             var links = parser.Parse(source);
44             var formattedLinks = links.Format();
45             Assert.Equal(target, formattedLinks);
46         }
47
48         [Fact]
49         public static void QuotedReferencesWithSpacesTest()
50         {

```

```

51     var source = @"('a a': 'b b' '"c c'")";
52     var target = @"('a a': 'b b' 'c c')";
53     var parser = new Parser();
54     var links = parser.Parse(source);
55     var formattedLinks = links.Format();
56     Assert.Equal(target, formattedLinks);
57 }
58
59 [Fact]
60 public static void ParseSimpleReferenceTest()
61 {
62     var source = "test";
63     var parser = new Parser();
64     var links = parser.Parse(source);
65     Assert.NotNull(links);
66     Assert.Single(links);
67     // Simple reference creates a singlet link with null Id and one value
68     Assert.Null(links[0].Id);
69     Assert.NotNull(links[0].Values);
70     Assert.Single(links[0].Values!);
71     Assert.Equal("test", links[0].Values?[0].Id);
72     Assert.Null(links[0].Values?[0].Values);
73 }
74
75 [Fact]
76 public static void ParseReferenceWithColonAndValuesTest()
77 {
78     var source = "parent: child1 child2";
79     var parser = new Parser();
80     var links = parser.Parse(source);
81     Assert.NotNull(links);
82     Assert.Single(links);
83     Assert.Equal("parent", links[0].Id);
84     Assert.NotNull(links[0].Values);
85     Assert.Equal(2, links[0].Values!.Count);
86     Assert.Equal("child1", links[0].Values![0].Id);
87     Assert.Equal("child2", links[0].Values![1].Id);
88 }
89
90 [Fact]
91 public static void ParseMultilineLinkTest()
92 {
93     var source = "(parent: child1 child2)";
94     var parser = new Parser();
95     var links = parser.Parse(source);
96     Assert.NotNull(links);
97     Assert.Single(links);
98     Assert.Equal("parent", links[0].Id);
99     Assert.NotNull(links[0].Values);
100    Assert.Equal(2, links[0].Values!.Count);
101 }
102
103 [Fact]
104 public static void ParseValuesOnlyTest()
105 {
106     var source = ": value1 value2";
107     var parser = new Parser();
108     // Standalone ':' is now forbidden and should throw an exception
109     Assert.Throws<ParseException>(() => parser.Parse(source));
110 }
111
112 [Fact]
113 public static void SingletLinkTest()
114 {
115     // Test singlet link
116     var input = "(singlet)";
117     var result = new Parser().Parse(input);
118     Assert.Single(result);
119     Assert.Null(result[0].Id);
120     Assert.NotNull(result[0].Values);
121     Assert.Single(result[0].Values!);
122     Assert.Equal("singlet", result[0].Values?[0].Id);
123     Assert.Null(result[0].Values?[0].Values);
124 }
125
126 [Fact]
127 public static void ValueLinkTest()
128 {

```

```

129     // Test value link
130     var input = "(value1 value2 value3)";
131     var result = new Parser().Parse(input);
132     Assert.NotEmpty(result);
133 }
134
135 [Fact]
136 public static void QuotedReferencesWithSpecialCharsTest()
137 {
138     // Test quoted references
139     var input = @"("id with spaces": "value with spaces")";
140     var result = new Parser().Parse(input);
141     Assert.NotEmpty(result);
142 }
143
144 [Fact]
145 public static void SingleQuotedReferencesTest()
146 {
147     // Test single-quoted references
148     var input = "('id': 'value')";
149     var result = new Parser().Parse(input);
150     Assert.NotEmpty(result);
151 }
152
153 [Fact]
154 public static void ParseQuotedReferencesValuesOnlyTest()
155 {
156     var source = "\"has space\" 'has:colon'";
157     var parser = new Parser();
158     var links = parser.Parse(source);
159     Assert.NotNull(links);
160     Assert.Single(links);
161     Assert.Null(links[0].Id);
162     Assert.NotNull(links[0].Values);
163     Assert.Equal(2, links[0].Values!.Count);
164     Assert.Equal("has space", links[0].Values![0].Id);
165     Assert.Equal("has:colon", links[0].Values![1].Id);
166     var formatted = links.Format();
167     Assert.Equal("( 'has space' 'has:colon')", formatted);
168 }
169
170 [Fact]
171 public static void NestedLinksTest()
172 {
173     // Test nested links
174     var input = "(outer: (inner: value))";
175     var result = new Parser().Parse(input);
176     Assert.NotEmpty(result);
177 }
178
179 [Fact]
180 public static void HyphenatedIdentifiersTest()
181 {
182     // Test support for hyphenated identifiers like in BugTest1
183     var source = @"(conan-center-index: repository info)";
184     var parser = new Parser();
185     var links = parser.Parse(source);
186     var target = links.Format();
187     Assert.Equal(source, target);
188 }
189
190 [Fact]
191 public static void MultipleWordsInQuotesTest()
192 {
193     var source = @"("New York": city state)";
194     var parser = new Parser();
195     var links = parser.Parse(source);
196     var target = links.Format();
197     // Should preserve quotes for multi-word references
198     Assert.Contains("New York", target);
199 }
200
201 [Fact]
202 public static void SpecialCharactersInQuotesTest()
203 {
204     var input = @"("key:with:colons": "value(with)parenns")";
205     var result = new Parser().Parse(input);
206     Assert.NotEmpty(result);

```

```

207     input = @"('key with spaces': 'value: with special chars')";
208     result = new Parser().Parse(input);
209     Assert.NotEmpty(result);
210 }
211
212 [Fact]
213 public static void DeeplyNestedTest()
214 {
215     var input = "(a: (b: (c: (d: (e: value))))))";
216     var result = new Parser().Parse(input);
217     Assert.NotEmpty(result);
218 }
219
220 [Fact]
221 public static void SingleLineWithIdTest()
222 {
223     // Test single-line link with id
224     var input = "id: value1 value2";
225     var result = new Parser().Parse(input);
226     Assert.NotEmpty(result);
227 }
228
229 [Fact]
230 public static void SingleLineWithoutIdTest()
231 {
232     // Test link without id (single-line) - now forbidden
233     var input = ": value1 value2";
234     Assert.Throws<ParseException>(() => new Parser().Parse(input));
235 }
236
237 [Fact]
238 public static void MultilineWithoutIdTest()
239 {
240     // Test link without id (multi-line) - now forbidden
241     var input = "(: value1 value2)";
242     Assert.Throws<ParseException>(() => new Parser().Parse(input));
243 }
244
245 [Fact]
246 public static void SimpleRefTest()
247 {
248     var input = "simple ref";
249     var result = new Parser().Parse(input);
250     Assert.NotEmpty(result);
251 }
252
253 [Fact]
254 public static void MultiLineLinkWithIdTest()
255 {
256     var input = "(id: value1 value2)";
257     var result = new Parser().Parse(input);
258     Assert.NotEmpty(result);
259 }
260
261 [Fact]
262 public static void LinkWithoutIdMultiLineTest()
263 {
264     var input = "(: value1 value2)";
265     Assert.Throws<ParseException>(() => new Parser().Parse(input));
266 }
267
268 [Fact]
269 public static void SimpleReferenceParserTest()
270 {
271     var input = "hello";
272     var result = new Parser().Parse(input);
273     Assert.Single(result);
274     Assert.Null(result[0].Id);
275     Assert.NotNull(result[0].Values);
276     Assert.Single(result[0].Values!);
277     Assert.Equal("hello", result[0].Values?[0].Id);
278 }
279
280 [Fact]
281 public static void QuotedReferenceParserTest()
282 {
283     var input = "\"hello world\"";
284     var result = new Parser().Parse(input);
285 
```

```

286     Assert.Single(result);
287     Assert.Null(result[0].Id);
288     Assert.NotNull(result[0].Values);
289     Assert.Single(result[0].Values!);
290     Assert.Equal("hello world", result[0].Values?[0].Id);
291 }
292
293 [Fact]
294 public static void SingletLinkParserTest()
295 {
296     var input = "(singlet)";
297     var result = new Parser().Parse(input);
298     Assert.Single(result);
299     Assert.Null(result[0].Id);
300     Assert.NotNull(result[0].Values);
301     Assert.Single(result[0].Values!);
302     Assert.Equal("singlet", result[0].Values?[0].Id);
303     Assert.Null(result[0].Values?[0].Values);
304 }
305
306 [Fact]
307 public static void ValueLinkParserTest()
308 {
309     var input = "(a b c)";
310     var result = new Parser().Parse(input);
311     Assert.Single(result);
312     Assert.Null(result[0].Id);
313     Assert.Equal(3, result[0].Values?.Count);
314 }
315
316 [Fact]
317 public static void ParseQuotedReferencesTest()
318 {
319     var input = "(\"quoted id\": \"value with spaces\")";
320     var parser = new Parser();
321     var result = parser.Parse(input);
322
323     Assert.NotEmpty(result);
324     var formatted = result.Format();
325     Assert.Contains("quoted id", formatted);
326     Assert.Contains("value with spaces", formatted);
327 }
328
329 [Fact]
330 public static void LinkWithoutIdSingleLineTest()
331 {
332     // Test that standalone ':' is forbidden and should throw
333     var input = ": value1 value2";
334     var parser = new Parser();
335
336     // C# parser forbids this syntax (like JS/Rust)
337     Assert.Throws<ParseException>(() => parser.Parse(input));
338 }
339
340 [Fact]
341 public static void SingleLineLinkWithIdTest()
342 {
343     var input = "id: value1 value2";
344     var parser = new Parser();
345     var result = parser.Parse(input);
346
347     Assert.NotEmpty(result);
348     Assert.NotNull(result[0].Id);
349     Assert.Equal("id", result[0].Id);
350 }
351
352 [Fact]
353 public static void LinkWithIdTest()
354 {
355     var input = "(id: a b c)";
356     var parser = new Parser();
357     var result = parser.Parse(input);
358
359     Assert.Single(result);
360     Assert.Equal("id", result[0].Id);
361     Assert.Equal(3, result[0].Values?.Count);
362 }
363
364 [Fact]

```

```

365 public static void QuotedReferenceTest()
366 {
367     var input = "\"quoted value\"";
368     var parser = new Parser();
369     var result = parser.Parse(input);
370
371     Assert.NotEmpty(result);
372     Assert.NotNull(result[0].Values);
373     Assert.Single(result[0].Values!);
374     Assert.Equal("quoted value", result[0].Values![0].Id);
375 }
376
377 [Fact]
378 public static void SimpleReferenceTest()
379 {
380     var input = "simplereference";
381     var parser = new Parser();
382     var result = parser.Parse(input);
383
384     Assert.NotEmpty(result);
385 }
386
387 [Fact]
388 public static void SingleLineLinkTest()
389 {
390     var input = "myid: value1 value2";
391     var parser = new Parser();
392     var result = parser.Parse(input);
393
394     Assert.Single(result);
395     Assert.Equal("myid", result[0].Id);
396     Assert.Equal(2, result[0].Values?.Count);
397 }
398
399 [Fact]
400 public static void ValuesOnlyInParenthesesTest()
401 {
402     var input = "(value1 value2)";
403     var parser = new Parser();
404     var result = parser.Parse(input);
405
406     Assert.Single(result);
407     Assert.Null(result[0].Id);
408     Assert.Equal(2, result[0].Values?.Count);
409 }
410
411 [Fact]
412 public static void ParseValuesOnlyStandaloneColonTest()
413 {
414     // Test that standalone ':' is forbidden and should throw
415     var input = ": value1 value2";
416     var parser = new Parser();
417
418     // C# parser forbids this syntax (like JS/Rust)
419     Assert.Throws<ParseException>(() => parser.Parse(input));
420 }
421
422 [Fact]
423 public static void QuotedReferencesWithSpacesInLinkTest()
424 {
425     var input = "(id: \"value with spaces\")";
426     var parser = new Parser();
427     var result = parser.Parse(input);
428
429     Assert.Single(result);
430     Assert.Equal("id", result[0].Id);
431     Assert.Single(result[0].Values!);
432     Assert.Equal("value with spaces", result[0].Values?[0].Id);
433 }
434 }
435 }

```

### 1.38 ./Link.Foundation.Links.Notation.Tests/StreamParserTests.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Threading.Tasks;
5 using Xunit;
6

```

```

7 namespace Link.Foundation.Links.Notation.Tests
8 {
9     public static class StreamParserTests
10    {
11        private const string Document =
12            "first loves data\n" +
13            "# streamed comment\n" +
14            "profile:\n" +
15            "    name Ada\n" +
16            "    note \"line one\nline two\"\n" +
17            "(nested:\n    child value)\n" +
18            "last sees first";
19
20        private static string[] Render(IEnumerable<Link<string>> links) =>
21            links.Select(link => link.ToString()).ToArray();
22
23        [Fact]
24        public static void MatchesCanonicalParserOneSymbolAtATime()
25        {
26            var stream = new StreamParser();
27            foreach (var character in Document)
28            {
29                stream.Write(character.ToString());
30            }
31
32            Assert.Equal(Render(new Parser().Parse(Document)), Render(stream.Finish()));
33        }
34
35        [Fact]
36        public static void EmitsOnlyCompleteRecords()
37        {
38            var seen = new List<Link<string>>();
39            var stream = new StreamParser();
40            stream.LinkParsed += seen.Add;
41
42            Assert.Empty(stream.Write("profile:\n    name Ada\n    note \"first\nsecond\"\n"));
43            var emitted = stream.Write("n");
44
45            Assert.Single(emitted);
46            Assert.Equal(emitted, seen);
47        }
48
49        [Fact]
50        public static void SupportsLineChunksFinalRecordAndPosition()
51        {
52            var stream = new StreamParser();
53            foreach (var line in Document.SplitAfter('\n'))
54            {
55                stream.Write(line);
56            }
57
58            Assert.Equal(Render(new Parser().Parse(Document)), Render(stream.Finish()));
59            Assert.Equal(Document.Length, stream.Position.Offset);
60            Assert.Equal(0, stream.Position.Buffered);
61        }
62
63        [Fact]
64        public static void SupportsDrainResetAndBoundedMemory()
65        {
66            var seen = new List<Link<string>>();
67            var stream = new StreamParser { Collect = false, MaxBufferSize = 8 };
68            stream.LinkParsed += seen.Add;
69            for (var index = 0; index < 100; index++) stream.Write("a\n");
70
71            Assert.Empty(stream.Drain());
72            Assert.Single(stream.Finish());
73            Assert.Equal(100, seen.Count);
74
75            stream.Reset();
76            Assert.Throws<ArgumentException>(() => { stream.Write("123456789"); });
77        }
78
79        [Fact]
80        public static void ProvidesLazyAdapters()
81        {
82            var actual = StreamParser.ParseChunks(new[] { "one link\n", "two link" });
83
84            Assert.Equal(new[] { "(one link)", "(two link)" }, Render(actual));
85        }
86    }
87 }

```

```

86
87 [Fact]
88 public static void RejectsWritesAfterFinish()
89 {
90     var stream = new StreamParser();
91     stream.Finish("one");
92
93     Assert.Throws<InvalidOperationException>(() => stream.Write("two"));
94 }
95
96 [Fact]
97 public static async Task ProvidesAnAsyncEnumerableAdapter()
98 {
99     static async IEnumerable<string> Chunks()
100     {
101         yield return "one link\n";
102         await Task.Yield();
103         yield return "two link";
104     }
105
106     var actual = new List<Link<string>>();
107     await foreach (var link in StreamParser.ParseChunksAsync(
108         Chunks(),
109         cancellationTokens: TestContext.Current.CancellationTokens)) actual.Add(link);
110
111     Assert.Equal(new[] { "(one link)", "(two link)" }, Render(actual));
112 }
113 }
114 }
115
116 internal static class StreamingTestStringExtensions
117 {
118     public static IEnumerable<string> SplitAfter(this string value, char delimiter)
119     {
120         var start = 0;
121         for (var index = 0; index < value.Length; index++)
122         {
123             if (value[index] != delimiter) continue;
124             yield return value.Substring(start, index - start + 1);
125             start = index + 1;
126         }
127         if (start < value.Length) yield return value.Substring(start);
128     }
129 }

```

### 1.39 ./Link.Foundation.Links.Notation.Tests/TupleTests.cs

```

1 using System.Collections.Generic;
2 using Xunit;
3 using LinkType = Link.Foundation.Links.Notation.Link<string>;
4 using id = Link.Foundation.Links.Notation.id<string>;
5 using = Link.Foundation.Links.Notation. <string>;
6
7 namespace Link.Foundation.Links.Notation.Tests
8 {
9     public class TupleTests
10     {
11         [Fact]
12         public void TupleToLinkTest()
13         {
14             var source = @"(papa (lovesMama: loves mama))
15 (son lovesMama)
16 (daughter lovesMama)
17 (all (love mama))";
18             var parser = new Parser();
19             var links = parser.Parse(source);
20             var targetFromString = links.Format();
21
22             IList<LinkType> constructedLinks = new List<LinkType>()
23             {
24                 ("papa", ( )("lovesMama", "loves", "mama")),
25                 ("son", "lovesMama"),
26                 ("daughter", "lovesMama"),
27                 ("all", ("love", "mama")),
28             };
29             var targetFromTuples = constructedLinks.Format();
30             Assert.Equal(targetFromString, targetFromTuples);
31         }
32
33         [Fact]

```

```
34     public void NamedTupleToLinkTest()
35     {
36         var source = @"(papa (lovesMama: loves mama))
37 (son lovesMama)
38 (daughter lovesMama)
39 (all (love mama))";
40         var parser = new Parser();
41         var links = parser.Parse(source);
42         var targetFromString = links.Format();
43
44         IList<LinkType> constructedLinks = new List<LinkType>()
45         {
46             ("papa", ((id)"lovesMama", "loves", "mama")),
47             ("son", "lovesMama"),
48             ("daughter", "lovesMama"),
49             ("all", ("love", "mama")),
50         };
51         var targetFromTuples = constructedLinks.Format();
52         Assert.Equal(targetFromString, targetFromTuples);
53     }
54 }
55 }
```

## Index

./Link.Foundation.Links.Notation.Tests/ApiTests.cs, 27  
./Link.Foundation.Links.Notation.Tests/CommentsTests.cs, 30  
./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs, 32  
./Link.Foundation.Links.Notation.Tests/EmptyReferenceTests.cs, 34  
./Link.Foundation.Links.Notation.Tests/FormatConfigTests.cs, 36  
./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs, 38  
./Link.Foundation.Links.Notation.Tests/IndentedIdNestedValuesTests.cs, 39  
./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs, 40  
./Link.Foundation.Links.Notation.Tests/LinkTests.cs, 42  
./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs, 44  
./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs, 45  
./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs, 48  
./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs, 53  
./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs, 55  
./Link.Foundation.Links.Notation.Tests/NestedIndentationTests.cs, 56  
./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs, 57  
./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs, 60  
./Link.Foundation.Links.Notation.Tests/NestingDepthTests.cs, 62  
./Link.Foundation.Links.Notation.Tests/NestingLimitTests.cs, 63  
./Link.Foundation.Links.Notation.Tests/ParseErrorPositionTests.cs, 66  
./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs, 68  
./Link.Foundation.Links.Notation.Tests/StreamParserTests.cs, 73  
./Link.Foundation.Links.Notation.Tests/TupleTests.cs, 75  
./Link.Foundation.Links.Notation/Comments.cs, 1  
./Link.Foundation.Links.Notation/FormatConfig.cs, 2  
./Link.Foundation.Links.Notation/FormatOptions.cs, 2  
./Link.Foundation.Links.Notation/FurthestFailureTracer.cs, 3  
./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs, 3  
./Link.Foundation.Links.Notation/IListExtensions.cs, 5  
./Link.Foundation.Links.Notation/Link.cs, 7  
./Link.Foundation.Links.Notation/LinkFormatExtensions.cs, 11  
./Link.Foundation.Links.Notation/LinksGroup.cs, 13  
./Link.Foundation.Links.Notation/ParseException.cs, 15  
./Link.Foundation.Links.Notation/Parser.cs, 18  
./Link.Foundation.Links.Notation/Quotes.cs, 19  
./Link.Foundation.Links.Notation/StreamParseException.cs, 21  
./Link.Foundation.Links.Notation/StreamParser.cs, 22  
./Link.Foundation.Links.Notation/.cs, 25  
./Link.Foundation.Links.Notation/id.cs, 26